

Micro-Capstone 2 — Assistente de CRM com Memória de Cliente

Chatbot de CRM B2B (Vértice Serviços, fictícia) com memória persistente de cliente em SQLite, recuperação seletiva de contexto e statefulness entre sessões e reinícios — com comparação com e sem memória, cinco cenários multi-turno e 76 testes automatizados.

Autor: Carlos Santos

Data: 30/09/2026

Stack: Node.js 22.18+ · TypeScript · SQLite nativo (node:sqlite) · OpenAI gpt-4o-mini ou modo mock

Este documento: <https://github.com/carlossantosgit/micro-capstone-02-crm-customer-memory-pdf>

Código-fonte: repositório privado; os arquivos principais estão no Anexo B e a transcrição completa da execução, no Anexo A

Conteúdo

1. Parte 1 — Visão geral, evidências e critérios de aceitação (README)
2. Parte 2 — Arquitetura por módulo e modelo de dados
3. Parte 3 — Decisão 001: persistência em SQLite nativo
4. Parte 4 — Decisão 002: recuperação sem embeddings
5. Parte 5 — Decisão 003: preferências como memória
6. Parte 6 — Decisão 004: sessões anteriores complementam a memória
7. Anexo A — Transcrição completa da demonstração (npm run demo)
8. Anexo B — Código-fonte principal

Este PDF reúne os documentos do repositório e a saída da demonstração. Todos os números vêm de `results/demo-mock.md` ou da execução dos testes; nenhum foi digitado à mão. As respostas marcadas com [mock] são simuladas a partir do prompt montado (ver o aviso no início da Parte 1).

Parte 1 — Visão geral, evidências e critérios de aceitação (README)

Arquivo de origem: README.md

Módulo 03 — Engenharia de Contexto em IA (Coders University). Um chatbot de CRM B2B, via CLI, que **lembra de cada cliente entre sessões e reinícios** e, antes de cada resposta, **escolhe quais lembranças entram no contexto do modelo**. Guarda tudo, mas manda pouco.

Item: Stack

Detalhe: Node.js 22.18+ · TypeScript sem build · SQLite nativo (`node:sqlite`) · OpenAI `gpt-4o-mini` ou modo `mock`

Item: Testes

Detalhe: 76 testes, 76 passando (`npm test`), 12 requisitos do enunciado mapeados na §12

Item: Demonstração

Detalhe: 5 cenários multi-turno, 19/19 verificações atendidas (`npm run demo` → `results/demo-mock.md`)

Item: Enunciado

Detalhe: `docs/enunciado.md` · Decisões: `docs/decisoese/` · Detalhe por módulo: `docs/arquitetura.md`

Sobre as evidências. As transcrições publicadas são do **modo mock**. O mock não é um modelo: ele monta a resposta só com o que encontra no prompt (as seções de memória) e marca cada resposta com `[mock]`. Ele demonstra o **pipeline de memória**: o que foi guardado, recuperado, descartado e enviado. Não demonstra a redação de um modelo. A execução com o `gpt-4o-mini` real foi tentada em 30/09/2026, mas a chave disponível foi recusada pela OpenAI (401). A saída foi preservada como evidência do tratamento de falha em `results/falha-api-401.md`. Com uma chave válida, o comando é `LLM_MODE=openai npm run demo (§8)`.

1. Visão geral

Uma empresa de serviços B2B (aqui, a fictícia **Vértice Serviços**, que implanta e dá suporte a sistemas de gestão) quer um assistente que acompanhe o relacionamento com cada cliente ao longo do tempo: preferências, problemas anteriores, pedidos, solicitações e contexto comercial.

O problema tem dois lados, e o trabalho trata os dois:

1. **Lembrar** (statefulness). O que o cliente disse numa conversa precisa estar disponível na próxima, mesmo depois de fechar a aplicação. Sem isso, o cliente repete tudo a cada contato.
2. **Não lembrar demais** (recuperação seletiva). Enviar todo o histórico ao modelo custa tokens, dilui o que importa e produz respostas que citam coisas fora de propósito. Um exemplo do enunciado: o gosto do cliente por café aparecendo numa resposta sobre HTTP 500.

A solução separa **histórico** (tudo o que foi dito, guardado no banco) de **memória** (só o que vale lembrar, extraído, deduplicado e atualizado). Antes de cada resposta, uma função de relevância decide quais memórias entram no prompt, com score mínimo e teto de quantidade.

2. Arquitetura

```
src/
  app.ts entrada da CLI
  container.ts monta a aplicação (CLI, demo e testes usam o mesmo código)
  cli/ cli.ts (menu e conversa) · format.ts (blocos de debug) · lineReader.ts
  config/env.ts LLM_MODE, chave, modelo, limites de memória e de contexto
  database/ database.ts (node:sqlite) · migrations.ts (esquema versionado)
  repositories/ customer · session · message · memory · contextLog (todo o SQL)
  services/ chatService (pipeline) · memoryService · contextService · sessionService · summarizer
  memory/ extractors.ts (regras | LLM) · memoryPolicy.ts (portão, dedup, atualização)
  llm/ llmClient.ts (ÚNICO ponto que chama a API) · mockLlmClient.ts · prompts.ts
  utils/ relevance.ts (score) · text.ts (termos, conceitos) · withRetry.ts · misc.ts
  demo/runScenarios.ts os 5 cenários → results/demo-<modo>.md
tests/
  unit/ persistence · sessions · memoryWrite · retrieval · chatService
  e2e/ scenarios (cenários 1-4) · restart (cenário 5, dois processos)
```

Componente: CLI

Papel: Menu, identificação do cliente, conversa. Não contém regra de negócio.

Componente: Session Service

Papel: Cria a sessão; ao encerrar, gera título e resumo sem apagar mensagens; recupera e retoma sessões; fecha sessões que ficaram abertas por queda do processo.

Componente: Chat Service

Papel: Executa o pipeline de 10 passos (§7) e a comparação com/sem memória.

Componente: Memory Service

Papel: Recupera (scoring + portão + limite) e grava memórias (extração → política).

Componente: Context Service

Papel: Monta o prompt em seções nomeadas, dentro de um orçamento de tokens.

Componente: LLM Client

Papel: Único acesso ao modelo, com retentativa e classificação de erro. `LLM_MODE=mock` troca pelo cliente simulado.

O que veio do Micro-Capstone 01, que continua intacto: a stack (Node 22.18+, TS sem build, `node --test`, `tsc --noEmit`), o `withRetry`, a classificação de erros transitórios da OpenAI, o `loadConfig` com validação de faixa, o `.env.local` com `npm run env:setup`, a interface `LlmClient` com cliente roteirizado nos testes e a saída JSON com schema e fallback seguro. Do repositório da disciplina vieram o `node:sqlite`, a estimativa de tokens e a janela deslizante de mensagens. A tabela completa está em [docs/arquitetura.md](#).

3. Persistência

SQLite nativo do Node (`node:sqlite`), o mesmo módulo usado em aula. Nenhuma dependência nova. Os dados ficam num arquivo:

Arquivo: `data/crm.db`

Criado por: `npm start` / `npm run dev` (configurável com `DB_PATH`)

Arquivo: `data/demo-mock.db`, `data/demo-mock-cli.db`

Criado por: `npm run demo` (recriados a cada execução)

A pasta `data/` fica fora do git: são dados de uso, e a demonstração é regerada pelo script. O esquema é aplicado por migrações numeradas (`PRAGMA user_version`). Reabrir um banco não recria nada, e um banco antigo é atualizado em ordem (testado da v1 para a v2).

Tabela: `customers`

Conteúdo: id, nome, empresa, cargo, observações, criação, última interação

Tabela: `sessions`

Conteúdo: id, cliente, título, resumo, status (`active` / `closed`), início, última interação, fim

Tabela: `messages`

Conteúdo: id, sessão, cliente, role (`user` / `assistant`), conteúdo, timestamp, ordem

Tabela: `memories`

Conteúdo: id, cliente, tipo, assunto, conteúdo, importância, status (`active` / `superseded`), substituída por, sessão e mensagem de origem, origem (`rule_extractor` / `llm_extractor` / `manual`), menções, criação, atualização

Tabela: `memory_sources`

Conteúdo: cada sessão/mensagem que **criou ou reforçou** uma memória (proveniência completa)

Tabela: `context_log`

Conteúdo: para cada chamada ao modelo: memórias enviadas, memórias que o modelo declarou usar, tokens estimados e reais

A decisão, com as alternativas e pesos, está em `docs/decisooes/001`.

4. Estrutura de memória

Memória ≠ histórico. Todas as mensagens ficam em `messages`. Só uma parte vira linha em `memories`. Na demonstração, 14 mensagens do cliente geraram 7 memórias e 1 reforço; as demais eram cumprimentos, agradecimentos, perguntas pontuais ou repetições (§10).

Tipo: `preference`

Exemplo real da demonstração: "Cliente prefere receber os relatórios em PDF."

Importância base: 0,6

Tipo: `previous_problem`

Exemplo real da demonstração: Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação."

Importância base: 0,8

Tipo: `request`

Exemplo real da demonstração: Solicitação do cliente: "Solicitamos a inclusão de mais 10 usuários no plano." (*teste*)

Importância base: 0,6

Tipo: `relationship_context`

Exemplo real da demonstração: Contexto do relacionamento: "A renovação do nosso contrato é em dezembro..." (*teste*)

Importância base: 0,7

Tipo: `order_history`

Exemplo real da demonstração: Histórico de pedido: "Contratamos o módulo de BI em agosto, pedido 4521." (*teste*)

Importância base: 0,7

Tipo: `general_fact`

Exemplo real da demonstração: Fato informado pelo cliente: "Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite."

Importância base: 0,5

A importância ganha +0,1 quando o texto toca um assunto do negócio (conceito de domínio, §5).

Extração (depois de cada interação, só da mensagem do cliente). A resposta do assistente nunca vira memória, porque não é fato sobre o cliente. São dois extratores:

- **por regras** (modo mock, e fallback do outro): padrões explícitos por tipo, frase a frase, em `src/memory/extractors.ts`;
- **por LLM** (modo openai): o modelo propõe candidatas em JSON com schema. Se a API cair ou o JSON vier inválido, o extrator recua para as regras (testado).

Política de escrita (`src/memory/memoryPolicy.ts`, a mesma para os dois extratores):

Passo: 1. Portão

Regra: tipo válido; não é cumprimento; ≥ 2 termos de conteúdo **ou** um assunto de negócio; importância $\geq 0,3$

Resultado: descarta "Obrigado.", "Ok, valeu!", "Cliente gosta de café."

Passo: 2. Preferência com atributo reconhecido

Regra: compara **atributo e valor** (`src/memory/preferenceSlots.ts`)

Resultado: mesmo atributo e mesmo valor $\rightarrow$ **reforça** (deduplicação) · mesmo atributo com **outro valor** $\rightarrow$ a antiga vira **superseded** (atualização), e os atributos dela que a mensagem nova não mencionou são **levados** para a memória nova · atributos diferentes $\rightarrow$ **convivem**

Passo: 3. Demais memórias

Regra: mesmo tipo e similaridade de Dice $\geq 0,6$ entre os termos (sem o nome do cliente)

Resultado: **reforça** a existente: +1 menção, data nova, maior importância

Passo: 4. Criação

Regra: nenhum dos casos acima

Resultado: memória nova

Os atributos de preferência reconhecidos, com o valor normalizado:

Frase: "reuniões pela manhã" / "no período da manhã"

Atributo = valor: `meeting_time = periodo:manha` (as duas $\rightarrow$ repetição)

Frase: "reuniões depois das 14h" $\times$ "antes das 14h"

Atributo = valor: `meeting_time = depois:14:00  $\times$  antes:14:00` (mudança)

Frase: "depois das 14:30" $\times$ "depois das 14:45" $\cdot$ "14h" $\times$ "14:00"

Atributo = valor: minutos contam: `14:30  $\neq$  14:45` (mudança) $\cdot$ `14:00 = 14:00` (repetição)

Frase: "reuniões por videoconferência"

Atributo = valor: `meeting_mode = video` (convive com o horário)

Frase: "relatórios em Excel **por e-mail**"

Atributo = valor: `report_format = excel  $\cdot$  report_channel = email` (substitui "em PDF")

Frase: "**não** receber os relatórios em PDF"

Atributo = valor: `report_format = nao:pdf` (negação faz parte do valor)

Frase: "reuniões **sem** vídeo pela manhã" $\times$ "sem vídeo, **mas não** pela manhã"

Atributo = valor: a negação vale para o que vem depois dela e termina no primeiro valor de outro atributo: `meeting_mode = nao:video` com `meeting_time = periodo:manha  $\times$  nao:periodo:manha` (mudança)

O atributo é separado dos termos de busca de propósito: a busca descarta "antes", "depois" e "não" como stopwords, mas aqui essas palavras mudam o sentido. A primeira versão decidia por similaridade de palavras e por "assunto" (conjunto de conceitos). Quatro rodadas de revisão adversarial do Codex reproduziram os erros dessa abordagem e dos

primeiros ajustes, todos com regressão em `memoryWrite.test.ts`:

- PDF → Excel reforçava o PDF;
- "depois" → "antes das 14h" era tratado como repetição;
- "por videoconferência" **apagava** "pela manhã";
- "Excel por e-mail" **não** substituí a PDF;
- "Excel por e-mail" → "PDF" perdia o envio por e-mail (agora a memória nova diz "... em PDF. Mantido da preferência anterior: envio dos relatórios por e-mail.");
- "14:30" → "14:45" era tratado como repetição (os minutos eram descartados);
- uma negação valia para a frase inteira e, depois, para todo valor seguinte no mesmo trecho ("sem vídeo pela manhã" chegou a ser consolidado como "não pela manhã").

No modo openai, se o modelo deixar de devolver uma informação só porque ela já existe, o detector por regras registra a reafirmação mesmo assim. Ele só reforça memória existente; nunca cria. Como as regras não cobrem toda formulação, a proteção dos resumos de sessões anteriores não depende só disso (§5).

5. Recuperação

Antes de cada resposta, **cada memória ativa** do cliente recebe um score em relação à mensagem atual:

```
score = 0,60 · textual + 0,25 · importância + 0,15 · recência
```

```
textual = max( conceitos em comum / conceitos da mensagem , termos da mensagem achados na memória / termos da mensagem )
```

```
recência = 0,5 ^ (dias desde a última atualização / 30)
```

- **Termos:** palavras sem acento, sem stopwords e sem palavras genéricas do domínio ("problema", "cliente", "sistema"), reduzidas a um radical simples. Se as palavras genéricas contassem, todo problema antigo pareceria relevante para todo problema novo.
- **Conceitos de domínio:** 11 grupos de sinônimos (`src/utils/text.ts`), que ligam o que a palavra sozinha não liga: **login** ↔ **autenticação** ↔ **SSO**, **relatório** ↔ **PDF** ↔ **planilha**, **reunião** ↔ **agenda** ↔ **horário**, **integração** ↔ **API** ↔ **webhook** ↔ **ERP**...

Portão e limite. Uma memória só entra se $\text{textual} \geq 0,25$ e $\text{score} \geq 0,45$ (`MIN_MEMORY_SCORE`), com no máximo 5 (`MAX_MEMORIES`). Importância e recência **ordenam**, mas nunca colocam sozinhas uma memória no contexto. Uma memória de importância 1,0 e atualizada hoje, sem relação com a pergunta, fica de fora (testado). É a regra contra o uso forçado aplicada **antes** do modelo, em vez de depender só da obediência dele.

Interações anteriores relacionadas. Os resumos de sessões encerradas recebem o mesmo critério textual ($0,7 \cdot \text{textual} + 0,3 \cdot \text{recência}$), com no máximo 2 (`MAX_PAST_SESSIONS`). Uma sessão só entra se trazer algo que a memória **não** cobre. Se ela é fonte de uma memória sobre o assunto da pergunta, fica de fora: com a memória ativa, a informação seria **redundante**; com a memória substituída, seria **obsoleta**. Há uma segunda camada, que não depende da proveniência: se o resumo trata do **mesmo atributo** de uma preferência memorizada (ex.: horário de reunião), a sessão também fica de fora, e quem responde por esse atributo é a memória, com o valor mais recente. Essa camada cobre as frases que a extração não reconheceu. As duas regras nasceram de defeitos achados pelos testes e pela revisão adversarial do Codex ([docs/decisoes/004](#)).

Por que não embeddings. O caso difícil do enunciado ("login" × "autenticação") se resolve com conceitos de domínio. Assim o score fica explicável (o debug mostra `casou conceito:autenticacao`), determinístico nos testes e sem chamada extra por mensagem. O risco dessa escolha é o sinônimo fora da lista, que gera um falso negativo. A troca por embeddings é local, só na função `textualRelevance` ([docs/decisoes/002](#)).

6. Statefulness

O estado do cliente vive **no arquivo SQLite, não no processo**. Nada que precise sobreviver fica em variável:

Momento: Mensagem do cliente chega

O que é gravado: a mensagem, **antes** de chamar o modelo (se a API cair, ela não se perde)

Momento: Modelo responde

O que é gravado: a resposta, a data da última interação e o registro do contexto (`context_log`)

Momento: Depois da resposta

O que é gravado: as memórias extraídas (criadas, reforçadas ou substituídas)

Momento: `/sair`

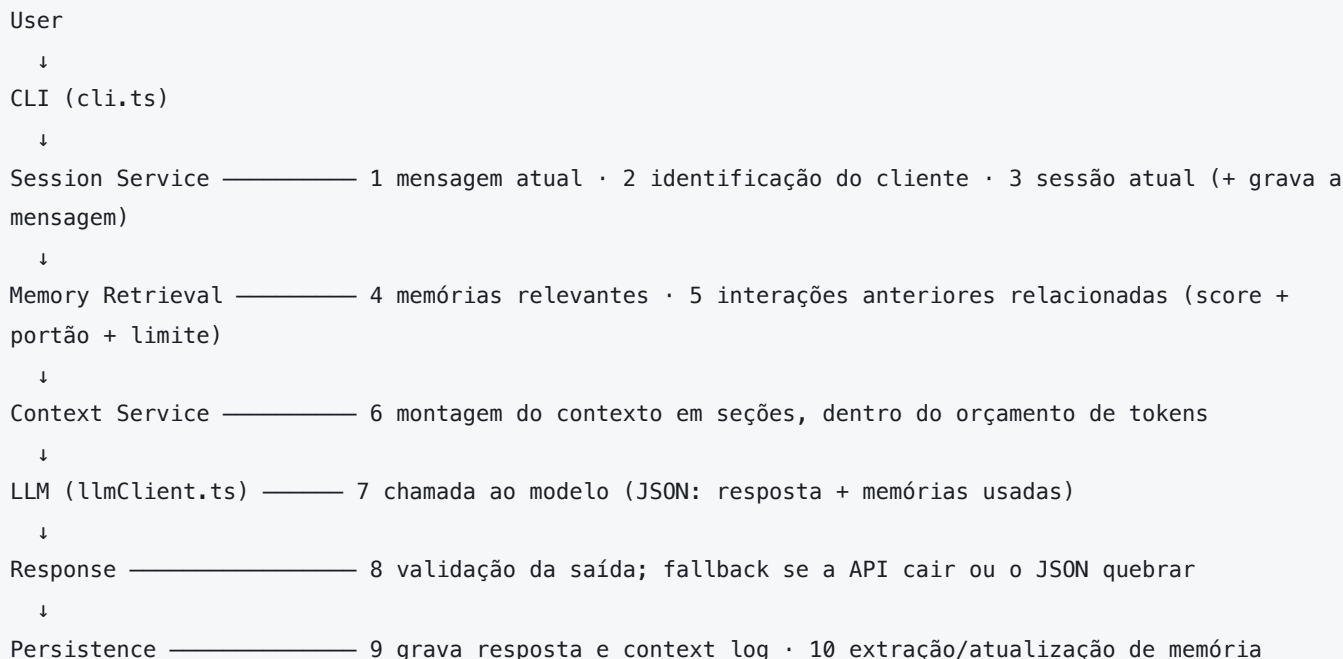
O que é gravado: título e resumo da sessão; status `closed` ; as mensagens continuam lá

Momento: Processo morre no meio do atendimento

O que é gravado: nada se perde. Na próxima sessão do mesmo cliente, a sessão órfã é encerrada e resumida automaticamente

Evidência de reinício real (`tests/e2e/restart.test.ts` e cenário 5 do demo): a CLI roda em **dois processos Node distintos**, e o segundo, sem nada em comum com o primeiro além do arquivo, encontra o cliente, a sessão anterior e a memória. Um segundo teste corta a entrada no meio do atendimento, sem `/sair`, e confirma que a mensagem e a memória sobreviveram.

7. Fluxo



Cada passo aparece numerado no bloco `[PIPELINE]` do modo debug (§10). O código está em `src/services/chatService.ts`, no método `handleMessage`.

O prompt montado, em seções nomeadas (`contextService.ts`):

```
system: # SYSTEM INSTRUCTIONS regras do assistente (inclui: usar memória só se melhorar a resposta)
      # CURRENT CUSTOMER nome, empresa, cargo, data de hoje
      # RELEVANT CUSTOMER MEMORY [M1] (preference · 02/09/2026) Cliente prefere receber os
relatórios em PDF.
      # RELATED PAST INTERACTIONS [S1] (15/09/2026) título – resumo (só se houver)
user/assistant: CURRENT SESSION as últimas 6 mensagens da sessão atual (HISTORY_MESSAGES)
user: CURRENT MESSAGE
```

Orçamento (CONTEXT_TOKEN_BUDGET , padrão 1.200 tokens estimados): se o total passar, o corte segue esta ordem: interações anteriores de menor score, depois memórias de menor score, depois as mensagens mais antigas da sessão. A instrução e a mensagem atual nunca são cortadas (testado).

8. Como executar

```
cd "Módulo 03 – Engenharia de Contexto em IA/micro-capstone-02-crm-customer-memory"
npm install

npm start # CLI, modo mock (padrão): sem API, sem custo
npm run dev # a mesma CLI com os blocos [PIPELINE] [MEMORY] [CONTEXT] [LLM] [MEMORY WRITE]
npm run demo # os 5 cenários com datas simuladas → results/demo-mock.md
```

Com o modelo real (gpt-4o-mini):

```
npm run env:setup # cria .env.local a partir do .env.example (fica fora do git)
# edite .env.local: OPENAI_API_KEY=sk-...
LLM_MODE=openai npm run dev # ou LLM_MODE=openai no próprio .env.local
LLM_MODE=openai npm run demo # → results/demo-openai.md, com tokens e custo medidos
```

No modo openai , a extração de memória e o resumo de sessão também passam a usar o modelo (MEMORY_EXTRACTOR=auto). Com MEMORY_EXTRACTOR=rules , só a resposta usa o modelo. As demais variáveis, com os valores padrão, estão em `.env.example`.

Menu da CLI:

```
1 – Novo atendimento identifica (ou cadastra) o cliente e abre uma sessão
2 – Recuperar atendimento lista as sessões, mostra data/resumo/mensagens/memórias; pode retomar
3 – Listar clientes sessões, memórias ativas e última interação de cada um
4 – Ver memória de um cliente ativas (com importância, menções, origem) e o histórico das substituídas
5 – Comparar resposta com e sem memória a mesma pergunta nas duas condições; nada é gravado
0 – Sair
```

Na conversa: `/sair` encerra a sessão (com resumo), `/memoria` mostra a memória do cliente e `/debug` liga ou desliga os detalhes.

9. Como executar os testes

```
npm test # 76 testes, offline (sem API; o LLM é o mock ou um roteiro)
npm run typecheck # tsc --noEmit
```


Resultado em 30/09/2026: `i tests 76 · i pass 76 · i fail 0; tsc` sem erros. Dezesesseis desses testes são regressões dos achados de quatro rodadas de revisão adversarial do Codex (`memoryWrite.test.ts` › "regressões da ... revisão adversarial do Codex"). As correções foram validadas por mutação: com cada proteção desligada, as regressões correspondentes falham. Cada teste usa um banco SQLite **em disco**, num diretório temporário apagado no fim, porque persistência é justamente o que está sendo testado. Nenhum teste lê `.env.local` nem chama a API.

10. Demonstração multi-turno

Transcrição completa, gerada por `npm run demo: results/demo-mock.md`. Um único cliente (Carlos Mendes, Logística Andrade) acumula sessões de 02/09 a 30/09/2026. As datas são simuladas pelo relógio injetável. Isso deixa a memória mais difícil de usar bem: a cada pergunta, há de 5 a 6 memórias ativas competindo, e só a relevante deve entrar.

Histórico que cria a memória (trechos literais):

```
--- sessão · 02/09/2026 ---
Carlos: Bom dia! · não virou memória (cumprimento/agradecimento)
Carlos: Prefiro receber os relatórios em PDF. + criada (preference, importância 0.7): Cliente prefere receber os relatórios em PDF.
Carlos: Obrigado. · não virou memória (cumprimento/agradecimento)
--- sessão · 08/09/2026 ---
Carlos: Prefiro reuniões pela manhã. + criada (preference, importância 0.7): Cliente prefere reuniões pela manhã.
Carlos: Gosto muito de café especial. + criada (preference, importância 0.6): Cliente gosta de café especial.
--- sessão · 10/09/2026 ---
Carlos: Prefiro fazer reuniões no período da manhã.
      = deduplicada: reforça mem_... (similaridade 1, agora 2 menções): Cliente prefere reuniões pela manhã.
```

Cenário 1 — Preferência (15/09, sessão nova: "Pode me enviar o relatório mensal?")

```
[MEMORY] 1 memória(s) recuperada(s) de 5 ativa(s) (score mínimo 0.45, máximo 5)
type: preference · score: 0.89 – casou conceito:relatorio · content: Cliente prefere receber os relatórios em PDF.
Descartadas: previous_problem 0.34, previous_problem 0.34, preference 0.31, preference 0.28 → sem relação com a mensagem (textual 0.00)
Assistente: [mock] Carlos, considereei o seu histórico: vou seguir a sua preferência registrada em 02/09/2026
              (Cliente prefere receber os relatórios em PDF). ...
```

Cenário 2 — Problema anterior (22/09: "Estamos com problema de login novamente.")

```
[1] previous_problem · score 0.93 – casou conceito:autenticacao · "Tivemos um problema de autenticação na implantação."
[2] previous_problem · score 0.93 – casou conceito:autenticacao · "O SSO da equipe financeira não aceitava as credenciais."
Descartadas: as 3 preferências (reunião, PDF, café) → textual 0.00
[MEMORY WRITE] + criada (previous_problem): "Estamos com problema de login novamente." ← a recorrência também fica registrada
```

Nenhuma palavra de "login novamente" aparece em "autenticação na implantação". A ligação é feita pelo conceito de domínio.

Cenário 3 — Informação irrelevante (24/09: "Qual o status do chamado 123?", seguida de "Minha API está retornando HTTP 500.")

```
[MEMORY] 0 memória(s) recuperada(s) de 6 ativa(s)
- preference · score 0.28 · Cliente prefere reuniões pela manhã. → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.25 · Cliente gosta de café especial. → sem relação com a mensagem (textual 0.00 < 0.25)
  (... e os 3 problemas e a preferência de PDF, pelo mesmo motivo)
Assistente: [mock] Olá, Carlos. Recebi a sua mensagem: "Qual o status do chamado 123?". Não tenho registro anterior sobre esse assunto; ...
```

Nenhuma dessas memórias entrou no contexto das duas mensagens: o teste do cenário 3 confere o texto do prompt, e o demo confere a seleção e a resposta. O assistente também não inventa um status para o chamado; a instrução proíbe fatos fora do contexto.

Cenário 4 — Alteração de preferência (26/09: "Agora prefiro reuniões depois das 14h."; 30/09: "Podemos agendar uma reunião para revisar o projeto?")

```
26/09 [MEMORY WRITE] v atualizada: mem_... "Cliente prefere reuniões pela manhã." → substituída por mem_... "Cliente prefere reuniões depois das 14h."
30/09 [MEMORY] 1 memória recuperada · preference · score 0.91 · Cliente prefere reuniões depois das 14h.
    Assistente: [mock] Carlos, considere o seu histórico: vou seguir a sua preferência registrada em 26/09/2026
        (Cliente prefere reuniões depois das 14h). ...
Memória do cliente (menu 4):
  Histórico de memórias substituídas (1) – não entram no contexto:
    - [preference] Cliente prefere reuniões pela manhã. → substituída por mem_... em 26/09/2026
```

Cenário 5 — Persistência após reinício (CLI real, dois processos: `node src/app.ts --debug`)

```
# processo 1
> Marina Souza → Cliente "Marina Souza" não encontrado. Cadastrar agora? s ...
Marina: Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite.
+ criada mem_... (general_fact, importância 0.6): Fato informado pelo cliente: "Nosso ERP é o SAP e ..."
Marina: /sair · Escolha: 0 ← o processo termina

# processo 2 (outro pid, mesmo arquivo SQLite)
> marina → Cliente encontrado. · Última interação: 30/09/2026 · Sessões anteriores: 1 · Memórias registradas: 1
Marina: A integração com o ERP falhou esta noite, podem verificar?
[1] type: general_fact · score: 0.90 – casou conceito:integracao, termo:integr, termo:erp
Assistente: [mock] Marina, considere o seu histórico: há um registro relacionado de 30/09/2026:
    Fato informado pelo cliente: "Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite." ...
```

Verificações automáticas do demo (19 no total, todas atendidas):

Cenário: Deduplicação

Atendidas: 1/1

O que se verifica: a 2ª formulação reforçou em vez de duplicar

Cenário: 1

Atendidas: 4/4

O que se verifica: PDF recuperado · memórias sem relação fora · resposta cita PDF · sem memória, não cita

Cenário: 2

Atendidas: 3/3

O que se verifica: problema de autenticação recuperado por "login" · resposta o retoma · sem memória, não retoma

Cenário: 3

Atendidas: 4/4

O que se verifica: 0 memória para o chamado · resposta sem reunião/café · HTTP 500 sem café · comparação sem memória injetada

Cenário: 4

Atendidas: 4/4

O que se verifica: antiga marcada como substituída · só "14h" no contexto · prompt sem "manhã" · resposta usa 14h

Cenário: 5

Atendidas: 3/3

O que se verifica: dois processos distintos sem erro · 1º criou a memória · 2º a recuperou

Uso do contexto na demonstração (contagem impressa pelo próprio script): 9 sessões e 28 mensagens gravadas (14 do cliente), das quais saíram 7 memórias e 1 reforço (6 ativas, 1 substituída). Nos 14 turnos de atendimento, o prompt teve ~523 tokens estimados em média (máximo ~605, orçamento 1.200), e foram enviadas **7 memórias no total, 0,5 por turno**, com até 6 ativas disponíveis. O histórico não vai inteiro ao modelo: a sessão atual entra pela janela das últimas 6 mensagens, e as anteriores entram só como memória ou resumo filtrados.

11. Com memória x sem memória

`npm run dev` → opção 5, ou `app.chatService.compare()`. A mesma pergunta vai ao modelo duas vezes, **com o estado da memória no momento em que a pergunta chega**. Nada é gravado.

Condição: Sem memória

O modelo recebe: instrução + sessão atual + mensagem atual

Condição: Com memória

O modelo recebe: instrução + cliente + **memórias relevantes** + **interações anteriores relevantes** + sessão atual + mensagem atual

Resultado nas quatro perguntas finais do demo (`results/demo-mock.md` , blocos `[COMPARAÇÃO]`):

Pergunta: "Pode me enviar o relatório mensal?"

Memórias enviadas (de ativas): 1 de 5 (PDF, 0,89)

Tokens sem → com: ~423 → ~496

Resposta sem memória: "Não tenho registro anterior sobre esse assunto... me passe mais detalhes"

Resposta com memória: segue a preferência por PDF

Pergunta: "Estamos com problema de login novamente."
Memórias enviadas (de ativas): 2 de 5 (autenticação, SSO; 0,93)
Tokens sem → com: ~425 → ~556
Resposta sem memória: pede detalhes
Resposta com memória: retoma os dois incidentes anteriores

Pergunta: "Qual o status do chamado 123?"
Memórias enviadas (de ativas): 0 de 6
Tokens sem → com: ~421 → ~484
Resposta sem memória: pede detalhes
Resposta com memória: igual: nenhuma memória era relevante

Pergunta: "Podemos agendar uma reunião...?"
Memórias enviadas (de ativas): 1 de 6 (14h, 0,91)
Tokens sem → com: ~428 → ~499
Resposta sem memória: pede detalhes
Resposta com memória: usa "depois das 14h", não "manhã"

O cenário 3 é o controle. A condição "com memória" ganha só o bloco do cliente (37 tokens) e a seção de memória vazia (25), juntos ~63 tokens, e **nenhuma** memória. A resposta não muda. A memória aparece onde ajuda e some onde não ajuda. O acréscimo de contexto ficou entre ~63 e ~131 tokens estimados por pergunta.

Trecho literal (cenário 1):

```
=====
SEM MEMÓRIA
=====
(contexto: instrução + sessão atual + mensagem · ~423 tokens)
[mock] Olá. Recebi a sua mensagem: "Pode me enviar o relatório mensal?". Não tenho registro anterior
sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

=====
COM MEMÓRIA
=====
(contexto: + cliente, memórias relevantes e interações anteriores · ~496 tokens)
Memórias utilizadas:
- [preference] Cliente prefere receber os relatórios em PDF. (score 0.89)
Descartadas pelo filtro: 4 de 5 ativa(s) (previous_problem 0.34, previous_problem 0.34, preference
0.31, preference 0.28)
O modelo declarou ter usado: M1
[mock] Carlos, considere o seu histórico: vou seguir a sua preferência registrada em 02/09/2026
(Cliente prefere receber os relatórios em PDF). Com isso, dou andamento à sua mensagem: "Pode me
enviar o relatório mensal?".
```

Limite desta evidência. No mock, a diferença entre as colunas vem inteiramente do contexto, e é isso que ela prova: o que chega ao modelo em cada condição. A qualidade da redação de um modelo real com esse contexto **não foi medida** (a execução real falhou por chave recusada, ver o topo). A regra contra o uso forçado tem duas camadas. A primeira, o portão de relevância, é determinística e está testada. A segunda, a instrução ao modelo ("use uma memória somente quando ela melhorar diretamente a resposta"), só se verifica com o modelo real.

12. Critérios de aceitação

Critérios obrigatórios do enunciado

Critério: A memória realmente é utilizada nas respostas

Onde está: `contextService` põe as memórias selecionadas no prompt; o modelo devolve `memorias_usadas`, gravado em `context_log`

Evidência: `chatService.test.ts` inspeciona o prompt enviado e confere a citação; demo, cenários 1, 2, 4 e 5

Critério: Existe persistência real

Onde está: SQLite em arquivo (`node:sqlite`), migrações versionadas

Evidência: `persistence.test.ts` (nova conexão, migração v1→v2)

Critério: Os dados sobrevivem ao reinício

Onde está: nada que precise sobreviver fica em variável; mensagem gravada antes do modelo

Evidência: `restart.test.ts`: dois processos Node e entrada cortada no meio; demo, cenário 5

Critério: Clientes podem possuir múltiplas sessões

Onde está: `sessions.customer_id`; menu 1 sempre abre uma nova

Evidência: `persistence.test.ts` "cria sessão ativa..."; demo: 9 sessões de um cliente

Critério: Sessões anteriores podem ser recuperadas

Onde está: menu 2: lista, detalhes (data, resumo, últimas mensagens, memórias) e retomada

Evidência: `sessions.test.ts` (5 testes)

Critério: Interações são armazenadas

Onde está: `messages` (user e assistant, em ordem) e `context_log`

Evidência: `persistence.test.ts` "cada turno grava..."

Critério: Informações relevantes podem virar memória

Onde está: extração por regras ou LLM, depois da resposta

Evidência: `memoryWrite.test.ts` §5 (seis tipos, exemplos do enunciado)

Critério: Memórias irrelevantes não são utilizadas

Onde está: portão de relevância (textual $\geq 0,25$, score $\geq 0,45$) antes do modelo

Evidência: `retrieval.test.ts` §8 (chamado 123; HTTP 500 x café); demo, cenário 3

Critério: Existe deduplicação

Onde está: Dice $\geq 0,6$ no mesmo tipo → reforço (+menção), com proveniência em `memory_sources`

Evidência: `memoryWrite.test.ts` §9 (exemplo do enunciado); demo, 10/09

Critério: Memória pode ser atualizada

Onde está: mesmo atributo de preferência, valor novo → `superseded` + `superseded_by`, preservando os atributos não mencionados

Evidência: `memoryWrite.test.ts` §10 e regressões (PDF → Excel, 14h → 15h, depois → antes, 14:30 → 14:45, com → sem, negação por trecho, PDF → Excel por e-mail → PDF); `scenarios.test.ts` cenário 4

Critério: O contexto enviado ao modelo é limitado

Onde está: máx. 5 memórias, 2 sessões anteriores, 6 mensagens recentes e orçamento de 1.200 tokens com corte ordenado

Evidência: `retrieval.test.ts` "contexto limitado" (3 testes) e §7; demo: 0,5 memória/turno

Critério: Existe comparação com e sem memória

Onde está: menu 5; `chatService.compare()`

Evidência: `chatService.test.ts` §12 (os dois prompts inspecionados; nada gravado); §11 acima

Critério: Existem exemplos multi-turno

Onde está: `npm run demo` → `results/demo-mock.md`

Evidência: 5 cenários, 19/19 verificações; `scenarios.test.ts`

Critério: Existem testes

Onde está: `tests/unit`, `tests/e2e`

Evidência: 76/76 (§9)

Critério: Existe documentação completa

Onde está: este README, `docs/arquitetura.md`, `docs/decisoes/001-004`

Evidência: —

Critério: O Micro-Capstone 01 permanece funcionando

Onde está: nenhum arquivo dele foi alterado

Evidência: em 30/09/2026, no MC01: `git status` limpo, `npm test` 82/82, `npm run typecheck` sem erros

Os 12 testes pedidos

#: 1

Teste pedido: criação de cliente

Arquivo › suíte: `unit/persistence.test.ts` › 1. criação de cliente

#: 2

Teste pedido: criação de sessão

Arquivo › suíte: `unit/persistence.test.ts` › 2. criação de sessão

#: 3

Teste pedido: armazenamento de mensagens

Arquivo › suíte: `unit/persistence.test.ts` › 3. armazenamento de mensagens

#: 4

Teste pedido: recuperação de sessão

Arquivo › suíte: `unit/sessions.test.ts` › 4. recuperação de sessão

#: 5

Teste pedido: criação de memória

Arquivo › suíte: `unit/memoryWrite.test.ts` › 5. criação de memória

#: 6

Teste pedido: busca por memória

Arquivo › suíte: `unit/retrieval.test.ts` › 6. busca por memória

#: 7

Teste pedido: filtro de relevância

Arquivo › suíte: `unit/retrieval.test.ts` › 7. filtro de relevância

#: 8

Teste pedido: memória irrelevante sendo descartada

Arquivo › suíte: `unit/memoryWrite.test.ts` › 8 (na escrita) e `unit/retrieval.test.ts` › 8 (no contexto)

#: 9

Teste pedido: deduplicação

Arquivo › suíte: `unit/memoryWrite.test.ts` › 9. deduplicação

#: 10

Teste pedido: atualização de memória

Arquivo › suíte: `unit/memoryWrite.test.ts` › 10. atualização de memória

#: 11

Teste pedido: persistência após reinício

Arquivo › suíte: `unit/persistence.test.ts` › 11 (nova conexão) e `e2e/restart.test.ts` (dois processos)

#: 12

Teste pedido: comparação com e sem memória

Arquivo › suíte: `unit/chatService.test.ts` › 12. comparação com e sem memória

Tratamento de falhas

Situação: API do modelo indisponível

Comportamento: resposta de fallback, mensagem do cliente preservada, memória ainda extraída (por regras)

Evidência: `chatService.test.ts`; execução real com chave recusada em [results/falha-api-401.md](#)

Situação: Nenhuma memória encontrada

Comportamento: seção "(nenhuma memória relevante para esta mensagem)"; resposta normal

Evidência: `chatService.test.ts` "cliente novo"

Situação: Cliente sem sessões / sessão inexistente

Comportamento: mensagem clara ("ainda não possui sessões", "Sessão ... não encontrada")

Evidência: `sessions.test.ts`

Situação: Banco vazio

Comportamento: listagens vazias; o banco é criado na 1ª execução

Evidência: `persistence.test.ts`

Situação: Erro de parsing da resposta do modelo

Comportamento: texto puro é aproveitado; JSON quebrado vira fallback; na extração, recua para as regras

Evidência: `chatService.test.ts`, `memoryWrite.test.ts`

Situação: Configuração inválida (`LLM_MODE=openai` sem chave)

Comportamento: uma linha de erro, com a orientação `npm run env:setup` ou `LLM_MODE=mock`

Evidência: `src/config/env.ts`

A CLI mostra só a mensagem do erro. O stack trace aparece apenas no modo debug.

Limitações e decisões técnicas

- **Evidência de modelo real ausente.** Todo o pipeline está testado e demonstrado. Falta medir como o `gpt-4o-mini` redige com esse contexto, inclusive se ele obedece à instrução de não citar memória fora de propósito quando uma memória marginal passa no portão. O comando está pronto (`LLM_MODE=openai npm run demo`) e publica tokens e custo junto com as verificações.
- **Cobertura dos conceitos de domínio.** Sinônimos fora dos 11 grupos não se ligam. Ex.: "não consigo entrar no portal" não encontra o problema de autenticação. O erro resultante é a omissão (resposta sem histórico), não a injeção de memória errada.
- **Extrator por regras é conservador.** Ele reconhece padrões explícitos ("prefiro", "tivemos um problema", "contratamos"...) e deixa passar formulações indiretas. No modo `openai`, a extração é feita pelo modelo, com as regras como rede de segurança.
- **Atributos de preferência são uma lista fechada** (horário e modalidade de reunião, formato e canal de relatório, canal de contato). Uma preferência fora dela (ex.: idioma do atendimento, "em inglês" → "em português") não é substituída: as duas convivem e o modelo recebe as duas com as datas. A regra 3 da instrução manda valer a mais recente, mas isso depende do modelo.

- **Substituição só para preferências.** Um problema novo sobre o mesmo assunto vira uma memória nova (registro de recorrência), e não substituição. É o comportamento desejado para incidentes, mas significa que "o problema de login foi resolvido" não apaga a memória do problema: não há tipo "resolução".
- **Uma sessão com dois assuntos** é excluída inteira das interações anteriores quando um deles já está na memória (decisão 004).
- **Tokens estimados** (1 token $\approx$ 3 caracteres, como em aula). No modo openai, o `context_log` guarda também os tokens reais (`usage`).
- **Observações do cadastro não vão ao prompt**, por decisão (003). O que deve orientar respostas precisa passar pela memória.

Parte 2 — Arquitetura por módulo e modelo de dados

Arquivo de origem: docs/arquitetura.md

Complementa o README (§2 e §7). Aqui ficam o mapa de arquivos, o modelo de dados completo e o que foi reaproveitado do Micro-Capstone 01 e do repositório da disciplina.

Mapa de módulos

Camada: Entrada

Arquivo: src/app.ts

Responsabilidade: Lê a configuração, monta a aplicação e abre a CLI. Erro de configuração vira mensagem de uma linha.

Camada:

Arquivo: src/container.ts

Responsabilidade: Monta tudo a partir da AppConfig . CLI, demonstração e testes usam esta mesma função.

Camada: CLI

Arquivo: src/cli/cli.ts

Responsabilidade: Menu (1–5, 0), identificação do cliente, laço de conversa, /sair , /memoria , /debug . Não contém regra.

Camada:

Arquivo: src/cli/format.ts

Responsabilidade: Blocos de transparência: [PIPELINE] , [MEMORY] , [PAST SESSIONS] , [CONTEXT] , [LLM] , [MEMORY WRITE] .

Camada:

Arquivo: src/cli/lineReader.ts

Responsabilidade: Leitura de linhas que funciona no terminal e por pipe (permite roteirizar a CLI nos testes).

Camada: Configuração

Arquivo: src/config/env.ts

Responsabilidade: LLM_MODE , chave, modelo, limites de recuperação e orçamento, com validação de faixa (padrão do MC01).

Camada: Banco

Arquivo: src/database/database.ts

Responsabilidade: Conexão node:sqlite , foreign_keys , migração e transação.

Camada:

Arquivo: src/database/migrations.ts

Responsabilidade: Esquema versionado (v1: tabelas principais; v2: memory_sources).

Camada: Repositórios

Arquivo: src/repositories/*.ts

Responsabilidade: Todo o SQL, um arquivo por tabela. Nenhum serviço escreve SQL.

Camada: Serviços

Arquivo: src/services/chatService.ts

Responsabilidade: O pipeline de 10 passos e a comparação com/sem memória.

Camada:

Arquivo: src/services/memoryService.ts

Responsabilidade: Recuperação seletiva (memórias e sessões anteriores) e escrita pela política.

Camada:

Arquivo: `src/services/contextService.ts`

Responsabilidade: Montagem do prompt em seções nomeadas, dentro do orçamento de tokens.

Camada:

Arquivo: `src/services/sessionService.ts`

Responsabilidade: Ciclo da sessão: criar, encerrar com resumo, detalhar, retomar, fechar sessões órfãs.

Camada:

Arquivo: `src/services/summarizer.ts`

Responsabilidade: Resumo da sessão (por LLM no modo openai; por regras no mock e como fallback).

Camada: Memória

Arquivo: `src/memory/extractors.ts`

Responsabilidade: Extração de candidatas (por regras ou por LLM, que recua para as regras quando falha).

Camada:

Arquivo: `src/memory/memoryPolicy.ts`

Responsabilidade: Portão, deduplicação e atualização: funções puras.

Camada:

Arquivo: `src/memory/preferenceSlots.ts`

Responsabilidade: Atributo e valor de uma preferência (horário/modalidade de reunião, formato/canal de relatório, canal de contato), com operador e negação.

Camada: LLM

Arquivo: `src/llm/llmClient.ts`

Responsabilidade: Único ponto que chama a API (OpenAI), com retentativa e classificação de erro.

Camada:

Arquivo: `src/llm/mockLlmClient.ts`

Responsabilidade: Cliente simulado que só "sabe" o que está no prompt.

Camada:

Arquivo: `src/llm/prompts.ts`

Responsabilidade: Instruções e schemas JSON (resposta, extração, resumo).

Camada: Utilitários

Arquivo: `src/utils/relevance.ts`

Responsabilidade: Fórmula do score, portão e limite.

Camada:

Arquivo: `src/utils/text.ts`

Responsabilidade: Normalização, radicais, stopwords, conceitos de domínio, Dice, detecção de cumprimento.

Camada: Demonstração

Arquivo: `src/demo/runScenarios.ts`

Responsabilidade: Os 5 cenários com datas simuladas → `results/demo-<modo>.md`.

Dependência entre camadas: `cli` → `services` → (`memory`, `repositories`, `llm`) → `database`. Os serviços recebem os repositórios, o cliente LLM e o relógio por injeção. Os testes trocam o LLM por um roteiro e o relógio por datas simuladas, sem mock de banco.

Modelo de dados

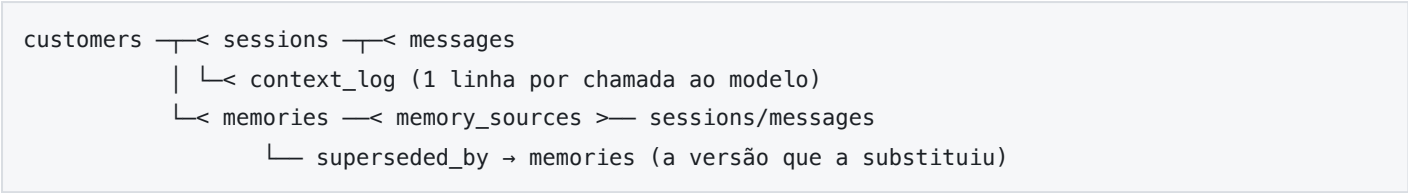


Tabela: customers
Colunas principais: id, name, company, role, notes, created_at, last_interaction_at
Observação: notes não vai ao prompt (decisão 003).

Tabela: sessions
Colunas principais: id, customer_id, title, summary, status (active / closed), started_at, last_interaction_at, ended_at
Observação: Resumo e título gerados ao encerrar.

Tabela: messages
Colunas principais: id, seq, session_id, customer_id, role (user / assistant), content, created_at
Observação: seq garante a ordem dentro da sessão.

Tabela: memories
Colunas principais: id, customer_id, type, topic, content, importance, status (active / superseded), superseded_by, source_session_id, source_message_id, origin, mentions, created_at, updated_at
Observação: CHECK nos seis tipos do enunciado.

Tabela: memory_sources
Colunas principais: memory_id, session_id, message_id, action (created / reinforced), created_at
Observação: Proveniência completa (decisão 004).

Tabela: context_log
Colunas principais: id, session_id, message_id, with_memory, memory_ids, memories_cited, est_tokens, prompt_tokens, completion_tokens, llm
Observação: Evidência auditável de cada contexto enviado.

Reaproveitamento

De onde: Micro-Capstone 01
O quê: Stack: Node 22.18+, TypeScript sem build, node --test , tsc --noEmit , openai ^6.45
Como ficou: Idêntica. tsconfig.json copiado sem mudança.

De onde: Micro-Capstone 01
O quê: src/utils/withRetry.ts
Como ficou: Copiado sem mudança no corpo.

De onde: Micro-Capstone 01
O quê: isTransientOpenAIError , retryAfterMs , maxRetries: 0 no SDK
Como ficou: Mesmo desenho, em src/llm/llmClient.ts .

De onde: Micro-Capstone 01
O quê: loadConfig com readNumber validado, requireApiKey , .env.local + env:setup
Como ficou: Mesmo padrão, acrescido de LLM_MODE e dos limites de memória.

De onde: Micro-Capstone 01
O quê: Interface LlmClient + cliente roteirizado nos testes
Como ficou: Mesmo padrão (ScriptedLlm em tests/helpers.ts).

De onde: Micro-Capstone 01

O quê: Saída JSON com schema (Structured Outputs) e fallback seguro

Como ficou: Resposta `{resposta, memorias_usadas}`, com fallback quando a API cai ou o JSON quebra.

De onde: Repositório da disciplina

O quê: `messageRepository.ts` com `node:sqlite`

Como ficou: Mesmo módulo, agora com esquema versionado e quatro entidades.

De onde: Repositório da disciplina

O quê: `estimateTokens` (1 token $\approx$ 3 caracteres)

Como ficou: Mesma estimativa, em `src/utils/misc.ts`.

De onde: Repositório da disciplina

O quê: `contextWindow` (janela deslizante) e `keywordScore`

Como ficou: Viraram `HISTORY_MESSAGES` e o termo textual do score.

Parte 3 — Decisão 001: persistência em SQLite nativo

Arquivo de origem: docs/decisoes/001-persistencia-sqlite-nativo.md

Data: 2026-09-30 · Estado: aceita

Contexto

O enunciado exige persistência real, que sobreviva ao reinício do processo, e prefere SQLite. O Micro-Capstone 01 não tem banco: ele avalia atendimentos sem estado. O repositório da disciplina (coders-university-pos-ia-m3-main/src/messageRepository.ts) já usa node:sqlite (DatabaseSync) para guardar mensagens.

Opções consideradas (critério · peso)

Critérios: sem dependência nova (0,3) · durabilidade real em disco (0,3) · simplicidade para quem avalia (0,2) · alinhamento com a aula (0,2).

Opção: node:sqlite (nativo)

Sem dependência: 1

Durável: 1

Simples: 1

Aula: 1

Total: 1,00

Opção: better-sqlite3

Sem dependência: 0 (binário nativo a compilar)

Durável: 1

Simples: 0,8

Aula: 0,5

Total: 0,56

Opção: Arquivo JSON

Sem dependência: 1

Durável: 0,5 (sem transação; escrita parcial corrompe)

Simples: 1

Aula: 0

Total: 0,65

Opção: DuckDB (usado na aula para vetores)

Sem dependência: 0,5 (já no projeto da aula, não no MC01)

Durável: 1

Simples: 0,6

Aula: 0,5

Total: 0,67

Decisão e evidência

node:sqlite, um arquivo data/crm.db, esquema em migrações numeradas (PRAGMA user_version). A evidência de que funciona como persistência real:

- `tests/e2e/restart.test.ts` roda a CLI em **dois processos Node distintos** e o segundo encontra a memória criada pelo primeiro;
- o mesmo teste corta a entrada no meio do atendimento (sem `/sair`) e nada se perde, porque cada mensagem é gravada no momento em que chega;
- `tests/unit/persistence.test.ts` migra um banco da versão 1 para a 2 preservando os dados.

O que se perde / risco da opção escolhida

- `node:sqlite` exige Node 22.13+ (o projeto já exige 22.18+ pelo TypeScript sem build, como no MC01). Em versões antigas o módulo simplesmente não existe.
- Acesso síncrono: numa API com muitos usuários simultâneos bloquearia o event loop. Para uma CLI de um operador isso não pesa.
- A busca por nome ignora acentos em JavaScript, não no SQL, e lê todos os nomes. Isso é barato com dezenas de clientes, mas não escala para milhares.

Sensibilidade: o que mudaria a decisão

Uso multiusuário concorrente (servidor HTTP) → PostgreSQL. Necessidade de busca semântica em volume → extensão vetorial ou DuckDB, como na aula.

Parte 4 — Decisão 002: recuperação sem embeddings

Arquivo de origem: docs/decisoões/002-recuperacao-lexical-com-conceitos.md

Data: 2026-09-30 · Estado: aceita

Contexto

O enunciado pede recuperação seletiva, com embeddings "somente se trouxer benefício claro". O desafio concreto é o cenário 2: "problema de **login**" precisa encontrar "problema de **autenticação**", e as duas frases não têm nenhuma palavra em comum.

Opções consideradas (critério · peso)

Critérios: explicabilidade do score (0,35) · resolve o cenário 2 (0,25) · sem custo/latência extra por mensagem (0,2) · determinismo nos testes (0,2).

Opção: Só palavras-chave (keywordScore da aula)

Explicável: 1

Cenário 2: 0

Sem custo: 1

Determinístico: 1

Total: 0,75

Opção: Termos + conceitos de domínio

Explicável: 1

Cenário 2: 1

Sem custo: 1

Determinístico: 1

Total: 1,00

Opção: Embeddings (text-embedding-3-small)

Explicável: 0,3 (cosseno sem justificativa legível)

Cenário 2: 1

Sem custo: 0 (1 chamada por memória nova e por mensagem)

Determinístico: 0,5

Total: 0,46

Opção: Híbrido (hybridScore da aula)

Explicável: 0,6

Cenário 2: 1

Sem custo: 0

Determinístico: 0,5

Total: 0,56

Decisão e evidência

$\text{score} = 0,60 \cdot \text{textual} + 0,25 \cdot \text{importância} + 0,15 \cdot \text{recência}$, com textual = o maior entre (conceitos em comum ÷ conceitos da pergunta) e (termos da pergunta achados ÷ termos da pergunta). Os conceitos são 11 grupos de sinônimos do domínio (src/utis/text.ts : autenticação = login, SSO, senha, 2FA...). Há um **portão**: textual $\geq 0,25$ e score $\geq 0,45$, com no máximo 5 memórias.

Evidência (`results/demo-mock.md`): "login" recuperou os dois problemas de autenticação com score 0,93 e o motivo casou `conceito:autenticacao` . As três preferências sem relação ficaram de fora com textual 0,00. Em "status do chamado 123", as 6 memórias ativas foram descartadas.

O que se perde / risco da opção escolhida

- **Cobertura limitada à lista de conceitos.** Um sinônimo fora dela ("não consigo entrar no portal") não liga "entrar" a "autenticação". Esse é o principal risco: um falso negativo, em que o assistente responde sem o histórico. É um erro mais barato que o oposto (injetar memória errada), mas é um erro.
- O radical de 6 letras é grosseiro: "contrato" e "contraste" colidem. Isso só pesa se a colisão coincidir com o assunto da memória.

Sensibilidade: o que mudaria a decisão

Com centenas de memórias por cliente, ou vocabulário que não cabe numa lista curta, embeddings passam a compensar. A troca é local: basta implementar `textualRelevance` com cosseno, sem mudar portão, limite nem o resto do pipeline.

Parte 5 — Decisão 003: preferências como memória

Arquivo de origem: docs/deciso0es/003-preferencias-como-memoria.md

Data: 2026-09-30 · Estado: aceita

Contexto

O enunciado lista "preferências" e "observações importantes" como campos do cliente. Se esses campos fossem sempre enviados na seção CURRENT CUSTOMER, pulariam o filtro de relevância: "gosta de café" iria ao prompt de toda pergunta sobre HTTP 500, exatamente o que a regra contra uso forçado proíbe.

Opções consideradas

1. Campos `preferences` / `notes` no cliente, sempre no prompt. É simples, mas viola a recuperação seletiva.
2. **Preferências como memórias do tipo** `preference`; `notes` **no cadastro, só para o operador**. Todo conteúdo que chega ao modelo passa pelo mesmo portão.
3. Não ter `notes`. Contraria o enunciado.

Decisão e evidência

Opção 2. A tabela `customers` tem `notes` (mostrado na opção 4 da CLI), mas o prompt recebe do cadastro só nome, empresa, cargo e a data de hoje. As preferências são memórias com assunto (`topic`), o que permite deduplicar e substituir (atualização). Evidência: `tests/unit/retrieval.test.ts` ("API retornando HTTP 500 não traz o gosto por café") e o cenário 3 em `results/demo-mock.md`.

O que se perde / risco da opção escolhida

Uma observação importante escrita só em `notes` nunca chega ao modelo. Se ela deve orientar respostas, o operador precisa registrá-la numa conversa, onde vira memória, ou o sistema ganharia uma "memória manual" pela CLI (o tipo de origem `manual` já existe no esquema, sem tela para isso).

Sensibilidade: o que mudaria a decisão

Se o negócio tiver uma informação que deve valer em **todo** atendimento (ex.: "cliente em risco de cancelamento"), ela merece um campo fixo e curto no CURRENT CUSTOMER, decidido explicitamente, e não uma lista livre de preferências.

Parte 6 — Decisão 004: sessões anteriores complementam a memória

Arquivo de origem: docs/decisoões/004-sessões-antiores-complementam-memória.md

Data: 2026-09-30 · Estado: aceita (nasceu de quatro defeitos: dois achados pelos testes, dois pelas revisões adversariais do Codex)

Contexto

O pipeline tem duas fontes de passado: **memórias** (fatos consolidados, com deduplicação e substituição) e **interações anteriores** (o resumo de sessões encerradas, pela seção RELATED PAST INTERACTIONS). Na primeira versão, as duas eram buscadas de forma independente.

O defeito que motivou a decisão

- Teste do cenário 4 (1ª falha).** O cliente trocou "reuniões pela manhã" por "depois das 14h". A memória antiga foi corretamente marcada como `superseded`, mas o **resumo da sessão antiga** ("Prefiro reuniões pela manhã") continuava textualmente relevante e voltava ao prompt. A informação invalidada entrava por outro caminho.
- Verificação automática do demo (2ª falha).** Depois da primeira correção, que ligava sessão a memória pela sessão de *origem*, a sessão de 10/09 ("Prefiro fazer reuniões no período da manhã") ainda vazava. Ela não criou memória: só **reforçou** a existente, por deduplicação, e por isso não aparecia como origem de nada.
- Revisão adversarial do Codex (3ª falha, no modo openai).** O prompt de extração mandava o modelo omitir o que já existia. Numa repetição, o modelo devolvia lista vazia, nada entrava em `memory_sources` e o vazamento voltava. Correção: o prompt agora pede as reafirmações, e o detector por regras registra as que o modelo omitir. Ele só reforça memória existente; nunca cria nem substitui.
- 2ª revisão do Codex (4ª falha).** A rede de segurança por regras não reconhece toda formulação ("Para reuniões, continuo preferindo o período da manhã"). Sem proveniência, o resumo dessa sessão voltava. Correção: uma 2ª camada, independente da proveniência, exclui a sessão cujo resumo trata do **mesmo atributo** de uma preferência memorizada (`preferenceSlots.ts`).

Opções consideradas

- Tirar as interações anteriores do prompt. Resolve o vazamento, mas perde contexto episódico que não vira memória (ex.: "você pediu a pauta da reunião trimestral") e contraria o enunciado ("contexto anterior relevante").
- Filtrar resumos por palavras de memórias substituídas. É frágil: depende da redação do resumo.
- Proveniência completa + regra de cobertura.** Uma tabela `memory_sources` registra cada sessão/mensagem que criou ou reforçou cada memória. Uma sessão anterior fica de fora se for fonte de uma memória relacionada à pergunta: ativa → **redundante** (a memória já está no contexto); substituída → **obsoleta**.

Decisão e evidência

Opção 3 (migração 2 do esquema). Evidência:

- `tests/unit/retrieval.test.ts`, suíte "interações anteriores relacionadas": a sessão episódica entra; a redundante e a obsoleta saem, com o motivo; a sessão que só reforçou também sai;
- `tests/e2e/scenarios.test.ts`, cenário 4: o prompt final não contém "manhã";

- `tests/unit/memoryWrite.test.ts`, "extrator por LLM que omite a repetição...": três sessões com o extrator roteirizado omitindo a repetição, e o prompt final não contém "manhã". Validado por mutação: sem a rede de segurança, o teste falha;
- `tests/unit/memoryWrite.test.ts`, "frase de repetição que as regras NÃO reconhecem...": sem proveniência nenhuma, a 2ª camada exclui o resumo. Validado por mutação;
- `results/demo-mock.md`: 19/19 verificações, incluindo "o prompt não contém mais a preferência pela manhã".

O que se perde / risco da opção escolhida

- Uma sessão que tratou de **dois** assuntos (um virou memória, o outro não) é excluída inteira quando a pergunta toca o assunto da memória. O custo é um falso negativo parcial.
- O resumo por regras cita as mensagens do cliente e pode carregar um assunto secundário da sessão. A instrução do modelo (regra 2) manda ignorar o que não ajuda, mas isso depende do modelo.

Sensibilidade: o que mudaria a decisão

Resumos por LLM mais disciplinados (um por assunto) permitiriam excluir só o trecho coberto, e não a sessão inteira.

Anexo A — Transcrição completa da demonstração (npm run demo)

Arquivo de origem: results/demo-mock.md

```
# Demonstração multi-turno – Micro-Capstone 02 (modo mock, modelo mock)
Gerado por `npm run demo` em 2026-09-30T18:43:59.394Z · banco data/demo-mock.db · datas das sessões simuladas
Parâmetros: MAX_MEMORIES=5 · MIN_MEMORY_SCORE=0.45 · HISTORY_MESSAGES=6 · CONTEXT_TOKEN_BUDGET=1200 · extrator rules
Modo mock: as respostas são simuladas ([mock]) a partir do prompt montado; o que se demonstra é o pipeline de memória, não a redação de um modelo.

#####
# Histórico do cliente (sessões anteriores que criam a memória)
#####

--- sessão ses_f1c454fbcd · 02/09/2026 ---
Carlos: Bom dia!
Assistente: [mock] Olá, Carlos. Recebi a sua mensagem: "Bom dia!". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.
    [MEMORY WRITE] extrator: rules
    · não virou memória (cumprimento/agradecimento): "Bom dia!"
Carlos: Prefiro receber os relatórios em PDF.
Assistente: [mock] Anotado, Carlos: vou registrar a sua preferência ("Prefiro receber os relatórios em PDF") e considerá-la nos próximos atendimentos.
    [MEMORY WRITE] extrator: rules
    + criada mem_b0c171731e (preference, importância 0.7): Cliente prefere receber os relatórios em PDF.
Carlos: Obrigado.
Assistente: [mock] Olá, Carlos. Recebi a sua mensagem: "Obrigado". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.
    [MEMORY WRITE] extrator: rules
    · não virou memória (cumprimento/agradecimento): "Obrigado."
(sessão encerrada · título: "Prefiro receber os relatórios em PDF")

--- sessão ses_1959459cd4 · 05/09/2026 ---
Carlos: Tivemos um problema de autenticação na implantação.
Assistente: [mock] Olá, Carlos. Recebi a sua mensagem: "Tivemos um problema de autenticação na implantação". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.
    [MEMORY WRITE] extrator: rules
    + criada mem_9cf703ef20 (previous_problem, importância 0.9): Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação."
Carlos: O SSO da equipe financeira não aceitava as credenciais.
Assistente: [mock] Carlos, considere o seu histórico: há um registro anterior relacionado, de 05/09/2026: Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação.". Com isso, dou andamento à sua mensagem: "O SSO da equipe financeira não aceitava as credenciais".
    [MEMORY WRITE] extrator: rules
```

+ criada mem_72538dcf12 (previous_problem, importância 0.9): Problema relatado pelo cliente: "O SSO da equipe financeira não aceitava as credenciais."
(sessão encerrada · título: "Tivemos um problema de autenticação na implantação...")

--- sessão ses_f491865405 · 08/09/2026 ---

Carlos: Prefiro reuniões pela manhã.

Assistente: [mock] Anotado, Carlos: vou registrar a sua preferência ("Prefiro reuniões pela manhã") e considerá-la nos próximos atendimentos.

[MEMORY WRITE] extrator: rules

+ criada mem_3a3a96a2a1 (preference, importância 0.7): Cliente prefere reuniões pela manhã.

Carlos: Gosto muito de café especial.

Assistente: [mock] Olá, Carlos. Recebi a sua mensagem: "Gosto muito de café especial". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

[MEMORY WRITE] extrator: rules

+ criada mem_d3dc9a8f51 (preference, importância 0.6): Cliente gosta de café especial.

(sessão encerrada · título: "Prefiro reuniões pela manhã")

--- sessão ses_6dfed2a041 · 10/09/2026 ---

Carlos: Prefiro fazer reuniões no período da manhã.

Assistente: [mock] Anotado, Carlos: vou registrar a sua preferência ("Prefiro fazer reuniões no período da manhã") e considerá-la nos próximos atendimentos.

[MEMORY WRITE] extrator: rules

= deduplicada: reforça mem_3a3a96a2a1 (similaridade 1, agora 2 menções): Cliente prefere reuniões pela manhã.

(sessão encerrada · título: "Prefiro fazer reuniões no período da manhã")

✓ verificação: segunda formulação da preferência de reunião reforçou a memória existente (sem duplicar)

Memórias ativas (5):

- [preference] Cliente prefere reuniões pela manhã.

importância 0.7 · 2 menção(ões) · criada 08/09/2026 · atualizada 10/09/2026 · origem rule_extractor · assunto reuniao · mem_3a3a96a2a1

- [preference] Cliente gosta de café especial.

importância 0.6 · 1 menção(ões) · criada 08/09/2026 · atualizada 08/09/2026 · origem rule_extractor · assunto - · mem_d3dc9a8f51

- [previous_problem] Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação."

importância 0.9 · 1 menção(ões) · criada 05/09/2026 · atualizada 05/09/2026 · origem rule_extractor · assunto autenticacao+implantacao · mem_9cf703ef20

- [previous_problem] Problema relatado pelo cliente: "O SSO da equipe financeira não aceitava as credenciais."

importância 0.9 · 1 menção(ões) · criada 05/09/2026 · atualizada 05/09/2026 · origem rule_extractor · assunto autenticacao · mem_72538dcf12

- [preference] Cliente prefere receber os relatórios em PDF.

importância 0.7 · 1 menção(ões) · criada 02/09/2026 · atualizada 02/09/2026 · origem rule_extractor · assunto relatorio · mem_b0c171731e

#####

Cenário 1 – Preferência

#####

Sessão inicial (02/09): "Prefiro receber os relatórios em PDF." · Nova sessão: "Pode me enviar o

relatório mensal?"

[COMPARAÇÃO – mesma pergunta, estado atual da memória, nada é gravado]

Pergunta: "Pode me enviar o relatório mensal?" · cliente: Carlos Mendes

=====

SEM MEMÓRIA

=====

(contexto: instrução + sessão atual + mensagem · ~423 tokens)

[mock] Olá. Recebi a sua mensagem: "Pode me enviar o relatório mensal?". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

=====

COM MEMÓRIA

=====

(contexto: + cliente, memórias relevantes e interações anteriores · ~496 tokens)

Memórias utilizadas:

– [preference] Cliente prefere receber os relatórios em PDF. (score 0.89)

Descartadas pelo filtro: 4 de 5 ativa(s) (previous_problem 0.34, previous_problem 0.34, preference 0.31, preference 0.28)

0 modelo declarou ter usado: M1

[mock] Carlos, considere o seu histórico: vou seguir a sua preferência registrada em 02/09/2026 (Cliente prefere receber os relatórios em PDF). Com isso, dou andamento à sua mensagem: "Pode me enviar o relatório mensal?".

[CONTEXT] sem memória · tokens/contexto aproximado: ~423 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~12

[CONTEXT] com memória · tokens/contexto aproximado: ~496 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~35 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~12

✓ verificação: sem memória, a resposta não sabe do PDF

--- sessão ses_8eaa00e410 · 15/09/2026 ---

Carlos: Pode me enviar o relatório mensal?

Assistente: [mock] Carlos, considere o seu histórico: vou seguir a sua preferência registrada em 02/09/2026 (Cliente prefere receber os relatórios em PDF). Com isso, dou andamento à sua mensagem: "Pode me enviar o relatório mensal?".

[PIPELINE]

1. mensagem recebida (34 caracteres)
2. cliente identificado: Carlos Mendes (cus_c5f8dd1296)
3. sessão atual ses_8eaa00e410: 0 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 5 ativa(s) → 1 relevante(s), 4 descartada(s)
5. interações anteriores: 4 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~496 tokens (orçamento 1200)
- 7–8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): nenhuma candidata

[MEMORY]

1 memória(s) recuperada(s) de 5 ativa(s) (score mínimo 0.45, máximo 5)

[1]

type: preference

score: 0.89 (textual 1.00 · importância 0.70 · recência 0.74) – casou conceito:relatorio, termo:relato
content: Cliente prefere receber os relatórios em PDF.

Descartadas (não entram no contexto):

- previous_problem · score 0.34 · Problema relatado pelo cliente: "Tivemos um problema de autenticação... → sem relação com a mensagem (textual 0.00 < 0.25)
- previous_problem · score 0.34 · Problema relatado pelo cliente: "O SSO da equipe financeira não aceit... → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.31 · Cliente prefere reuniões pela manhã. → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.28 · Cliente gosta de café especial. → sem relação com a mensagem (textual 0.00 < 0.25)

[PAST SESSIONS] 0 de 4 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~496 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~35 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~12

[LLM] ok · memórias que o modelo declarou usar: M1 · tokens reais: – (modo mock: sem consumo) · 0 ms

[MEMORY WRITE] extrator: rules

- não virou memória (pergunta sem fato durável): "Pode me enviar o relatório mensal?"

(sessão encerrada · título: "Pode me enviar o relatório mensal")

- ✓ verificação: a preferência por PDF foi recuperada para o contexto
- ✓ verificação: memórias sem relação (reunião, café, autenticação) ficaram fora
- ✓ verificação: a resposta usa a preferência (menciona PDF)

#####

Cenário 2 – Problema anterior

#####

Primeira sessão (05/09): "Tivemos um problema de autenticação na implantação." · Nova sessão: "Estamos com problema de login novamente."

[COMPARAÇÃO – mesma pergunta, estado atual da memória, nada é gravado]

Pergunta: "Estamos com problema de login novamente." · cliente: Carlos Mendes

=====

SEM MEMÓRIA

=====

(contexto: instrução + sessão atual + mensagem · ~425 tokens)

[mock] Olá. Recebi a sua mensagem: "Estamos com problema de login novamente". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

=====

COM MEMÓRIA

=====

(contexto: + cliente, memórias relevantes e interações anteriores · ~556 tokens)

Memórias utilizadas:

- [previous_problem] Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação." (score 0.93)
- [previous_problem] Problema relatado pelo cliente: "O SSO da equipe financeira não aceitava as credenciais." (score 0.93)

Descartadas pelo filtro: 3 de 5 ativa(s) (preference 0.29, preference 0.27, preference 0.26)

O modelo declarou ter usado: M1, M2

[mock] Carlos, considere o seu histórico: há um registro anterior relacionado, de 05/09/2026: Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação."; há um registro anterior relacionado, de 05/09/2026: Problema relatado pelo cliente: "O SSO da equipe financeira não aceitava as credenciais.". Com isso, dou andamento à sua mensagem: "Estamos com problema de login novamente".

[CONTEXT] sem memória · tokens/contexto aproximado: ~425 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~14

[CONTEXT] com memória · tokens/contexto aproximado: ~556 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~92 (2 itens) · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~14

✓ verificação: sem memória, a resposta não cita o incidente anterior

--- sessão ses_1afede4e6f · 22/09/2026 ---

Carlos: Estamos com problema de login novamente.

Assistente: [mock] Carlos, considere o seu histórico: há um registro anterior relacionado, de 05/09/2026: Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação."; há um registro anterior relacionado, de 05/09/2026: Problema relatado pelo cliente: "O SSO da equipe financeira não aceitava as credenciais.". Com isso, dou andamento à sua mensagem: "Estamos com problema de login novamente".

[PIPELINE]

1. mensagem recebida (40 caracteres)
2. cliente identificado: Carlos Mendes (cus_c5f8dd1296)
3. sessão atual ses_1afede4e6f: 0 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 5 ativa(s) → 2 relevante(s), 3 descartada(s)
5. interações anteriores: 5 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~556 tokens (orçamento 1200)
- 7-8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): 1 created

[MEMORY]

2 memória(s) recuperada(s) de 5 ativa(s) (score mínimo 0.45, máximo 5)

[1]

type: previous_problem

score: 0.93 (textual 1.00 · importância 0.90 · recência 0.68) – casou conceito:autenticacao

content: Problema relatado pelo cliente: "Tivemos um problema de autenticação na implantação."

[2]

type: previous_problem

score: 0.93 (textual 1.00 · importância 0.90 · recência 0.68) – casou conceito:autenticacao

content: Problema relatado pelo cliente: "O SSO da equipe financeira não aceitava as credenciais."

Descartadas (não entram no contexto):

– preference · score 0.29 · Cliente prefere reuniões pela manhã. → sem relação com a mensagem (textual 0.00 < 0.25)

– preference · score 0.27 · Cliente prefere receber os relatórios em PDF. → sem relação com a mensagem (textual 0.00 < 0.25)

– preference · score 0.26 · Cliente gosta de café especial. → sem relação com a mensagem (textual 0.00 < 0.25)

[PAST SESSIONS] 0 de 5 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~556 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~92 (2 itens) · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~14

[LLM] ok · memórias que o modelo declarou usar: M1, M2 · tokens reais: – (modo mock: sem consumo) · 0 ms

[MEMORY WRITE] extrator: rules

+ criada mem_887510b058 (previous_problem, importância 0.9): Problema relatado pelo cliente: "Estamos com problema de login novamente."

(sessão encerrada · título: "Estamos com problema de login novamente")

✓ verificação: o problema anterior de autenticação foi recuperado ('login' ↔ 'autenticação')

✓ verificação: a resposta retoma o problema anterior

#####

Cenário 3 – Informação irrelevante

#####

Memórias existentes: "prefere reuniões pela manhã" e "gosta muito de café especial". Pergunta: "Qual o status do chamado 123?"

[COMPARAÇÃO – mesma pergunta, estado atual da memória, nada é gravado]

Pergunta: "Qual o status do chamado 123?" · cliente: Carlos Mendes

=====

SEM MEMÓRIA

=====

(contexto: instrução + sessão atual + mensagem · ~421 tokens)

[mock] Olá. Recebi a sua mensagem: "Qual o status do chamado 123?". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

=====

COM MEMÓRIA

=====

(contexto: + cliente, memórias relevantes e interações anteriores · ~484 tokens)

Memórias utilizadas:

– (nenhuma memória passou no filtro de relevância para esta pergunta)

Descartadas pelo filtro: 6 de 6 ativa(s) (previous_problem 0.37, previous_problem 0.32, previous_problem 0.32, preference 0.28, preference 0.27, preference 0.25)

0 modelo declarou ter usado: nenhuma

[mock] Olá, Carlos. Recebi a sua mensagem: "Qual o status do chamado 123?". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

[CONTEXT] sem memória · tokens/contexto aproximado: ~421 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~10

[CONTEXT] com memória · tokens/contexto aproximado: ~484 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~25 (0 itens) · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~10

✓ verificação: na comparação, a condição 'com memória' não recebeu memória nenhuma

--- sessão ses_39195f7ae8 · 24/09/2026 ---

Carlos: Qual o status do chamado 123?

Assistente: [mock] Olá, Carlos. Recebi a sua mensagem: "Qual o status do chamado 123?". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

[MEMORY WRITE] extrator: rules

· não virou memória (pergunta sem fato durável): "Qual o status do chamado 123?"

Carlos: Minha API está retornando HTTP 500.

Assistente: [mock] Olá, Carlos. Recebi a sua mensagem: "Minha API está retornando HTTP 500". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

[PIPELINE]

1. mensagem recebida (35 caracteres)
2. cliente identificado: Carlos Mendes (cus_c5f8dd1296)
3. sessão atual ses_39195f7ae8: 2 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 6 ativa(s) → 0 relevante(s), 6 descartada(s)
5. interações anteriores: 6 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~559 tokens (orçamento 1200)
- 7–8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): nenhuma candidata

[MEMORY]

0 memória(s) recuperada(s) de 6 ativa(s) (score mínimo 0.45, máximo 5)

Descartadas (não entram no contexto):

– previous_problem · score 0.37 · Problema relatado pelo cliente: "Estamos com problema de login novame... → sem relação com a mensagem (textual 0.00 < 0.25)

– previous_problem · score 0.32 · Problema relatado pelo cliente: "Tivemos um problema de autenticação... → sem relação com a mensagem (textual 0.00 < 0.25)

– previous_problem · score 0.32 · Problema relatado pelo cliente: "O SSO da equipe financeira não aceit... → sem relação com a mensagem (textual 0.00 < 0.25)

– preference · score 0.28 · Cliente prefere reuniões pela manhã. → sem relação com a mensagem (textual 0.00 < 0.25)

- preference · score 0.27 · Cliente prefere receber os relatórios em PDF. → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.25 · Cliente gosta de café especial. → sem relação com a mensagem (textual 0.00 < 0.25)

[PAST SESSIONS] 0 de 6 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~559 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~25 (0 itens) · CURRENT SESSION ~73 (2 itens) · CURRENT MESSAGE ~12

[LLM] ok · memórias que o modelo declarou usar: nenhuma · tokens reais: - (modo mock: sem consumo) · 0 ms

[MEMORY WRITE] extrator: rules

· não virou memória (sem informação durável: pedido pontual, confirmação ou frase genérica): "Minha API está retornando HTTP 500."

(sessão encerrada · título: "Qual o status do chamado 123")

(detalhe da 1ª mensagem desta sessão)

[PIPELINE]

1. mensagem recebida (29 caracteres)
2. cliente identificado: Carlos Mendes (cus_c5f8dd1296)
3. sessão atual ses_39195f7ae8: 0 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 6 ativa(s) → 0 relevante(s), 6 descartada(s)
5. interações anteriores: 6 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~484 tokens (orçamento 1200)
- 7-8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): nenhuma candidata

[MEMORY]

0 memória(s) recuperada(s) de 6 ativa(s) (score mínimo 0.45, máximo 5)

Descartadas (não entram no contexto):

- previous_problem · score 0.37 · Problema relatado pelo cliente: "Estamos com problema de login novame... → sem relação com a mensagem (textual 0.00 < 0.25)
- previous_problem · score 0.32 · Problema relatado pelo cliente: "Tivemos um problema de autenticação... → sem relação com a mensagem (textual 0.00 < 0.25)
- previous_problem · score 0.32 · Problema relatado pelo cliente: "O SSO da equipe financeira não aceit... → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.28 · Cliente prefere reuniões pela manhã. → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.27 · Cliente prefere receber os relatórios em PDF. → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.25 · Cliente gosta de café especial. → sem relação com a mensagem (textual 0.00 < 0.25)

[PAST SESSIONS] 0 de 6 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~484 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~25 (0 itens) ·
CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~10
[LLM] ok · memórias que o modelo declarou usar: nenhuma · tokens reais: – (modo mock: sem consumo) · 0
ms

[MEMORY WRITE] extrator: rules

- não virou memória (pergunta sem fato durável): "Qual o status do chamado 123?"
- ✓ verificação: nenhuma memória entrou no contexto para 'status do chamado 123'
- ✓ verificação: a resposta não menciona reunião nem café
- ✓ verificação: 'API retornando HTTP 500' também não traz o café (regra contra uso forçado)

#####

Cenário 4 – Alteração de preferência

#####

Primeiro (08/09): "Prefiro reuniões pela manhã." · Depois (26/09): "Agora prefiro reuniões depois das 14h."

--- sessão ses_929870b06a · 26/09/2026 ---

Carlos: Agora prefiro reuniões depois das 14h.

Assistente: [mock] Anotado, Carlos: vou registrar a sua preferência ("Agora prefiro reuniões depois das 14h") e considerá-la nos próximos atendimentos.

[PIPELINE]

1. mensagem recebida (38 caracteres)
2. cliente identificado: Carlos Mendes (cus_c5f8dd1296)
3. sessão atual ses_929870b06a: 0 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 6 ativa(s) → 1 relevante(s), 5 descartada(s)
5. interações anteriores: 7 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~499 tokens (orçamento 1200)
- 7–8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): 1 superseded

[MEMORY]

1 memória(s) recuperada(s) de 6 ativa(s) (score mínimo 0.45, máximo 5)

[1]

type: preference

score: 0.88 (textual 1.00 · importância 0.70 · recência 0.69) – casou conceito:reuniao, termo:reunia

content: Cliente prefere reuniões pela manhã.

Descartadas (não entram no contexto):

- previous_problem · score 0.36 · Problema relatado pelo cliente: "Estamos com problema de login novame... → sem relação com a mensagem (textual 0.00 < 0.25)
- previous_problem · score 0.32 · Problema relatado pelo cliente: "Tivemos um problema de autenticação... → sem relação com a mensagem (textual 0.00 < 0.25)
- previous_problem · score 0.32 · Problema relatado pelo cliente: "O SSO da equipe financeira não aceit... → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.26 · Cliente prefere receber os relatórios em PDF. → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.25 · Cliente gosta de café especial. → sem relação com a mensagem (textual 0.00

< 0.25)

[PAST SESSIONS] 0 de 7 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~499 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~37 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~13

[LLM] ok · memórias que o modelo declarou usar: nenhuma · tokens reais: – (modo mock: sem consumo) · 0 ms

[MEMORY WRITE] extrator: rules

• atualizada: mem_3a3a96a2a1 "Cliente prefere reuniões pela manhã." → substituída por mem_0f4cd1be6a "Cliente prefere reuniões depois das 14h."

(sessão encerrada · título: "Agora prefiro reuniões depois das 14h")

✓ verificação: a memória antiga foi marcada como substituída pela nova

[COMPARAÇÃO – mesma pergunta, estado atual da memória, nada é gravado]

Pergunta: "Podemos agendar uma reunião para revisar o projeto?" · cliente: Carlos Mendes

=====

SEM MEMÓRIA

=====

(contexto: instrução + sessão atual + mensagem · ~428 tokens)

[mock] Olá. Recebi a sua mensagem: "Podemos agendar uma reunião para revisar o projeto?". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

=====

COM MEMÓRIA

=====

(contexto: + cliente, memórias relevantes e interações anteriores · ~499 tokens)

Memórias utilizadas:

– [preference] Cliente prefere reuniões depois das 14h. (score 0.91)

Descartadas pelo filtro: 5 de 6 ativa(s) (previous_problem 0.35, previous_problem 0.31, previous_problem 0.31, preference 0.25, preference 0.24)

O modelo declarou ter usado: M1

[mock] Carlos, considere o seu histórico: vou seguir a sua preferência registrada em 26/09/2026 (Cliente prefere reuniões depois das 14h). Com isso, dou andamento à sua mensagem: "Podemos agendar uma reunião para revisar o projeto?".

[CONTEXT] sem memória · tokens/contexto aproximado: ~428 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~17

[CONTEXT] com memória · tokens/contexto aproximado: ~499 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~33 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~17

--- sessão ses_483dbcb09 · 30/09/2026 ---

Carlos: Podemos agendar uma reunião para revisar o projeto?

Assistente: [mock] Carlos, considere o seu histórico: vou seguir a sua preferência registrada em 26/09/2026 (Cliente prefere reuniões depois das 14h). Com isso, dou andamento à sua mensagem: "Podemos agendar uma reunião para revisar o projeto?".

[PIPELINE]

1. mensagem recebida (51 caracteres)
2. cliente identificado: Carlos Mendes (cus_c5f8dd1296)
3. sessão atual ses_483dbcbc09: 0 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 6 ativa(s) → 1 relevante(s), 5 descartada(s)
5. interações anteriores: 8 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~499 tokens (orçamento 1200)
- 7-8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): nenhuma candidata

[MEMORY]

1 memória(s) recuperada(s) de 6 ativa(s) (score mínimo 0.45, máximo 5)

[1]

type: preference

score: 0.91 (textual 1.00 · importância 0.70 · recência 0.91) – casou conceito:reuniao, termo:reunia

content: Cliente prefere reuniões depois das 14h.

Descartadas (não entram no contexto):

- previous_problem · score 0.35 · Problema relatado pelo cliente: "Estamos com problema de login novame... → sem relação com a mensagem (textual 0.00 < 0.25)
- previous_problem · score 0.31 · Problema relatado pelo cliente: "Tivemos um problema de autenticação... → sem relação com a mensagem (textual 0.00 < 0.25)
- previous_problem · score 0.31 · Problema relatado pelo cliente: "O SSO da equipe financeira não aceit... → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.25 · Cliente prefere receber os relatórios em PDF. → sem relação com a mensagem (textual 0.00 < 0.25)
- preference · score 0.24 · Cliente gosta de café especial. → sem relação com a mensagem (textual 0.00 < 0.25)

[PAST SESSIONS] 0 de 8 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~499 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~37 · RELEVANT CUSTOMER MEMORY ~33 · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~17

[LLM] ok · memórias que o modelo declarou usar: M1 · tokens reais: – (modo mock: sem consumo) · 0 ms

[MEMORY WRITE] extrator: rules

- não virou memória (pergunta sem fato durável): "Podemos agendar uma reunião para revisar o projeto?"

(sessão encerrada · título: "Podemos agendar uma reunião para revisar o projet...")

✓ verificação: a única memória de reunião no contexto é a de depois das 14h

✓ verificação: o prompt não contém mais a preferência pela manhã

✓ verificação: a resposta usa a informação atualizada (14h)

Memórias ativas (6):

- [preference] Cliente prefere reuniões depois das 14h.
importância 0.7 · 1 menção(ões) · criada 26/09/2026 · atualizada 26/09/2026 · origem
rule_extractor · assunto reuniao · mem_0f4cd1be6a

- [previous_problem] Problema relatado pelo cliente: "Estamos com problema de login novamente."
importância 0.9 · 1 menção(ões) · criada 22/09/2026 · atualizada 22/09/2026 · origem
rule_extractor · assunto autenticacao · mem_887510b058

- [preference] Cliente gosta de café especial.
importância 0.6 · 1 menção(ões) · criada 08/09/2026 · atualizada 08/09/2026 · origem
rule_extractor · assunto - · mem_d3dc9a8f51

- [previous_problem] Problema relatado pelo cliente: "Tivemos um problema de autenticação na
implantação."
importância 0.9 · 1 menção(ões) · criada 05/09/2026 · atualizada 05/09/2026 · origem
rule_extractor · assunto autenticacao+implantacao · mem_9cf703ef20

- [previous_problem] Problema relatado pelo cliente: "O SSO da equipe financeira não aceitava as
credenciais."
importância 0.9 · 1 menção(ões) · criada 05/09/2026 · atualizada 05/09/2026 · origem
rule_extractor · assunto autenticacao · mem_72538dcf12

- [preference] Cliente prefere receber os relatórios em PDF.
importância 0.7 · 1 menção(ões) · criada 02/09/2026 · atualizada 02/09/2026 · origem
rule_extractor · assunto relatorio · mem_b0c171731e

Histórico de memórias substituídas (1) – não entram no contexto:

- [preference] Cliente prefere reuniões pela manhã. → substituída por mem_0f4cd1be6a em 26/09/2026

Cenário 5 – Persistência após reinício (CLI real, dois processos Node)
#####

\$ node src/app.ts --debug # processo 1 (pid 45795, saída 0)
Vértice Serviços · assistente de CRM com memória de cliente
modelo: mock · extrator de memória: rules · banco: data/demo-mock-cli.db · debug: ligado

CRM Assistant

1 – Novo atendimento
2 – Recuperar atendimento
3 – Listar clientes
4 – Ver memória de um cliente
5 – Comparar resposta com e sem memória
0 – Sair
Escolha: 1

Informe o cliente:

> Marina Souza

Cliente "Marina Souza" não encontrado. Cadastrar agora? (s/n) s

Empresa: Norte Engenharia

Cargo (opcional): Coordenadora de Operações

Observações importantes (opcional):

Cliente cadastrado: Marina Souza (cus_7b2316e0b6).

Última interação: –

Sessões anteriores: 0

Memórias registradas: 0 (a cada mensagem, só as relevantes entram no contexto)

Nova sessão criada:

ses_57805d84af

Digite a mensagem. Comandos: /sair encerra o atendimento · /memoria mostra a memória do cliente · /debug liga/desliga os detalhes.

Marina: Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite.

Assistente: [mock] Olá, Marina. Recebi a sua mensagem: "Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite". Não tenho registro anterior sobre esse assunto; vou registrar e dar andamento. Se puder, me passe mais detalhes.

[PIPELINE]

1. mensagem recebida (72 caracteres)
2. cliente identificado: Marina Souza (cus_7b2316e0b6)
3. sessão atual ses_57805d84af: 0 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 0 ativa(s) → 0 relevante(s), 0 descartada(s)
5. interações anteriores: 0 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~501 tokens (orçamento 1200)
- 7–8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): 1 created

[MEMORY]

0 memória(s) recuperada(s) de 0 ativa(s) (score mínimo 0.45, máximo 5)

[PAST SESSIONS] 0 de 0 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~501 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~41 · RELEVANT CUSTOMER MEMORY ~25 (0 itens) · CURRENT SESSION ~0 (0 itens) · CURRENT MESSAGE ~24

[LLM] ok · memórias que o modelo declarou usar: nenhuma · tokens reais: – (modo mock: sem consumo) · 0 ms

[MEMORY WRITE] extrator: rules

+ criada mem_5e3eb5d367 (general_fact, importância 0.6): Fato informado pelo cliente: "Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite."

Marina: /sair

Sessão ses_57805d84af encerrada. Histórico preservado.

Título: Nosso ERP é o SAP e a integração de pedidos roda...

Resumo: 2 mensagens (1 do cliente). Assuntos: integração, pedidos e contrato. Cliente: "Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite."

CRM Assistant


```
1 - Novo atendimento
2 - Recuperar atendimento
3 - Listar clientes
4 - Ver memória de um cliente
5 - Comparar resposta com e sem memória
0 - Sair
Escolha: 0
```

Até logo. Os dados ficam salvos em data/demo-mock-cli.db.

```
$ node src/app.ts --debug # processo 2 (pid 45882, saída 0) - nova execução, mesmo arquivo SQLite
Vértice Serviços · assistente de CRM com memória de cliente
modelo: mock · extrator de memória: rules · banco: data/demo-mock-cli.db · debug: ligado
```

CRM Assistant

```
1 - Novo atendimento
2 - Recuperar atendimento
3 - Listar clientes
4 - Ver memória de um cliente
5 - Comparar resposta com e sem memória
0 - Sair
Escolha: 1
```

```
Informe o cliente:
> marina
```

Cliente encontrado.

```
Última interação: 30/09/2026
Sessões anteriores: 1
Memórias registradas: 1 (a cada mensagem, só as relevantes entram no contexto)
```

```
Nova sessão criada:
ses_64e57dbbca
```

```
Digite a mensagem. Comandos: /sair encerra o atendimento · /memoria mostra a memória do cliente ·
/debug liga/desliga os detalhes.
```

Marina: A integração com o ERP falhou esta noite, podem verificar?

Assistente: [mock] Marina, considere o seu histórico: há um registro relacionado de 30/09/2026: Fato informado pelo cliente: "Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite.". Com isso, dou andamento à sua mensagem: "A integração com o ERP falhou esta noite, podem verificar?".

[PIPELINE]

1. mensagem recebida (58 caracteres)
2. cliente identificado: Marina Souza (cus_7b2316e0b6)
3. sessão atual ses_64e57dbbca: 0 mensagem(ns) recente(s) no histórico; mensagem do cliente gravada
4. memórias: 1 ativa(s) → 1 relevante(s), 0 descartada(s)

5. interações anteriores: 1 sessão(ões) resumida(s) → 0 relacionada(s)
6. contexto montado: ~527 tokens (orçamento 1200)
7-8. LLM mock: respondeu
9. resposta gravada e contexto registrado em context_log
10. memória (rules): 1 created

[MEMORY]

1 memória(s) recuperada(s) de 1 ativa(s) (score mínimo 0.45, máximo 5)

[1]

type: general_fact

score: 0.90 (textual 1.00 · importância 0.60 · recência 1.00) – casou conceito:integracao,
termo:integr, termo:erp

content: Fato informado pelo cliente: "Nosso ERP é o SAP e a integração de pedidos roda por webhook toda noite."

[PAST SESSIONS] 0 de 1 sessão(ões) anterior(es) resumida(s)

[CONTEXT] com memória · tokens/contexto aproximado: ~527 (orçamento 1200)

seções: SYSTEM INSTRUCTIONS ~411 · CURRENT CUSTOMER ~41 · RELEVANT CUSTOMER MEMORY ~55 · CURRENT
SESSION ~0 (0 itens) · CURRENT MESSAGE ~20

[LLM] ok · memórias que o modelo declarou usar: M1 · tokens reais: – (modo mock: sem consumo) · 0 ms

[MEMORY WRITE] extrator: rules

+ criada mem_af7a5e6592 (previous_problem, importância 0.9): Problema relatado pelo cliente: "A
integração com o ERP falhou esta noite, podem verificar?"

Marina: /sair

Sessão ses_64e57dbbca encerrada. Histórico preservado.

Título: A integração com o ERP falhou esta noite, podem v...

Resumo: 2 mensagens (1 do cliente). Assuntos: integração. Cliente: "A integração com o ERP falhou esta
noite, podem verificar?"

CRM Assistant

1 – Novo atendimento

2 – Recuperar atendimento

3 – Listar clientes

4 – Ver memória de um cliente

5 – Comparar resposta com e sem memória

0 – Sair

Escolha: 0

Até logo. Os dados ficam salvos em data/demo-mock-cli.db.

✓ verificação: os dois processos terminaram sem erro e são processos distintos

✓ verificação: o 1º processo criou a memória

✓ verificação: o 2º processo encontrou o cliente, a sessão anterior e a memória

#####

Resumo das verificações

#####

| Cenário | Verificações atendidas |

|---|---|

| Deduplicação | 1/1 |

| 1 | 4/4 |

| 2 | 3/3 |

| 3 | 4/4 |

| 4 | 4/4 |

| 5 | 3/3 |

Total: 19/19 verificações atendidas.

Chamadas ao modelo nos cenários 1–4 (sem contar o cenário 5): 22 · tokens: 0 prompt + 0 resposta
(mock: sem consumo)

Banco da demonstração (cenários 1–4): 9 sessões · 28 mensagens gravadas (14 do cliente) · 7 memórias criadas e 1 reforço(s) por deduplicação · 6 ativas e 1 substituída(s).

Contexto enviado ao modelo nos 14 turnos de atendimento: ~523 tokens em média (máximo ~605, orçamento 1200) · 7 memórias enviadas no total, ou seja, 0.5 por turno, contra até 6 ativas.

Anexo B — Código-fonte principal

Arquivos do repositório de código (privado), copiados sem edição; os espaços de alinhamento foram reduzidos para a extração de texto.

src/services/chatService.ts

```
// Chat Service – o pipeline de contexto, passo a passo:
//
// 1 mensagem atual (validação)
// 2 identificação do cliente (pela sessão)
// 3 recuperação da sessão atual + persistência da mensagem do cliente
// 4 busca de memórias relevantes ↳ scoring + portão + limite
// 5 busca de interações anteriores ↳ (filtragem de relevância)
// 6 montagem do contexto (orçamento de tokens)
// 7 LLM
// 8 resposta (validação do JSON; fallback se preciso)
// 9 persistência da resposta + registro do contexto usado
// 10 extração/atualização de memória (a partir da mensagem do cliente)
//
// A mensagem do cliente é gravada ANTES da chamada ao modelo: se a API cair, nada se perde.

import { InputError, UserFacingError } from "../domain/errors.ts"
import type { Customer, Memory, Message, Session } from "../domain/types.ts"
import type { LlmClient, TokenUsage } from "../llm/llmClient.ts"
import { describeLlmError } from "../llm/llmClient.ts"
import { FALLBACK_RESPONSE, RESPONSE_SCHEMA } from "../llm/prompts.ts"
import type { Repositories } from "../repositories/index.ts"
import type { Clock } from "../utils/misc.ts"
import { buildContext, type BuiltContext } from "../contextService.ts"
import type { ExtractionReport, MemoryService, RetrievalResult } from "../memoryService.ts"

export const MAX_MESSAGE_CHARS = 2000

export type ChatConfig = {
  historyMessages: number
  contextTokenBudget: number
  retrieval: { maxMemories: number; minScore: number; maxPastSessions: number }
  temperature?: number
}

export type Generation = {
  response: string
  // Rótulos ([M1], [S1]) que o modelo declarou ter usado, já validados contra o contexto.
  citedLabels: string[]
  fallbackReason: string | null
  parseWarning: string | null
  usage: TokenUsage | null
  latencyMs: number
}
```

```

export type TurnResult = {
  customer: Customer
  session: Session
  userMessage: Message
  assistantMessage: Message
  memories: RetrievalResult<Memory>
  pastSessions: RetrievalResult<Session>
  context: BuiltContext
  generation: Generation
  // IDs das memórias que o modelo declarou ter usado.
  citedMemoryIds: string[]
  extraction: ExtractionReport | { error: string }
  pipeline: string[]
}

export type ComparisonSide = { context: BuiltContext; generation: Generation }

export type ComparisonResult = {
  customer: Customer
  question: string
  memories: RetrievalResult<Memory>
  pastSessions: RetrievalResult<Session>
  withoutMemory: ComparisonSide
  withMemory: ComparisonSide
}

// Lê a saída do modelo. JSON inválido não derruba o atendimento: se vier texto puro, ele é
// usado como resposta; se não vier nada aproveitável, entra a resposta de fallback.
export function parseAssistantOutput(raw: string, validLabels: Set<string>): Pick<Generation,
"response" | "citedLabels" | "parseWarning" | "fallbackReason"> {
  const text = raw?.trim() ?? ""
  try {
    const data = JSON.parse(text) as { resposta?: unknown; memorias_usadas?: unknown }
    if (typeof data.resposta === "string" && data.resposta.trim()) {
      const cited = Array.isArray(data.memorias_usadas)
        ? [...new Set(data.memorias_usadas.filter((x): x is string => typeof x === "string").map((x)
=> x.replace(/[[\]\s]/g, "")))]
        : []
      return { response: data.resposta.trim(), citedLabels: cited, parseWarning: null, fallbackReason:
null }
    }
    return { response: FALLBACK_RESPONSE, citedLabels: [], parseWarning: 'JSON sem o campo
"resposta"', fallbackReason: "parse_error" }
  } catch {
    if (text && !text.startsWith("{")) return { response: text, citedLabels: [], parseWarning:
"resposta fora do formato JSON; texto usado como veio", fallbackReason: null }
    return { response: FALLBACK_RESPONSE, citedLabels: [], parseWarning: "JSON inválido ou vazio",
fallbackReason: "parse_error" }
  }
}

```

```

export type ChatDeps = { repos: Repositories; llm: LlmClient; memoryService: MemoryService; config:
ChatConfig; clock: Clock }

export class ChatService {
  private readonly deps: ChatDeps

  constructor(deps: ChatDeps) {
    this.deps = deps
  }

  get llmName(): string {
    return this.deps.llm.name
  }

  private validate(text: string): string {
    const message = (text ?? "").trim()
    if (!message) throw new InputError("Mensagem vazia: escreva a sua solicitação.")
    if (message.length > MAX_MESSAGE_CHARS) throw new InputError(`Mensagem acima de
${MAX_MESSAGE_CHARS} caracteres: resuma a sua solicitação.`)
    return message
  }

  private retrieveAll(customer: Customer, message: string, excludeSessionId?: string) {
    const { retrieval } = this.deps.config
    return {
      memories: this.deps.memoryService.retrieve(customer, message, { limit: retrieval.maxMemories,
minScore: retrieval.minScore }),
      pastSessions: this.deps.memoryService.retrievePastSessions(customer, message, { limit:
retrieval.maxPastSessions, excludeSessionId }),
    }
  }

  async generate(context: BuiltContext): Promise<Generation> {
    const started = Date.now()
    const labels = new Set(context.refs.map((r) => r.label))
    try {
      const completion = await this.deps.llm.complete({ messages: context.messages, temperature:
this.deps.config.temperature, json: RESPONSE_SCHEMA })
      return { ...parseAssistantOutput(completion.content, labels), usage: completion.usage ?? null,
latencyMs: Date.now() - started }
    } catch (error) {
      return { response: FALLBACK_RESPONSE, citedLabels: [], fallbackReason: `llm_error:
${describeLlmError(error)}`, parseWarning: null, usage: null, latencyMs: Date.now() - started }
    }
  }

  async handleMessage(sessionId: string, text: string): Promise<TurnResult> {
    const { repos, clock, config, memoryService } = this.deps
    const pipeline: string[] = []

```

```

// 1. mensagem atual
const message = this.validate(text)
pipeline.push(`1. mensagem recebida (${message.length} caracteres)`)

// 2-3. sessão atual e cliente
const session = repos.sessions.get(sessionId)
if (!session) throw new UserFacingError(`Sessão ${sessionId} não encontrada.`)
if (session.status !== "active") throw new UserFacingError(`A sessão ${sessionId} está encerrada.
Recupere-a (menu 2) para continuar.`)
const customer = repos.customers.get(session.customerId)
if (!customer) throw new UserFacingError(`Cliente da sessão ${sessionId} não encontrado.`)
pipeline.push(`2. cliente identificado: ${customer.name} (${customer.id})`)

const history = repos.messages.lastBySession(sessionId, config.historyMessages)
const userMessage = repos.messages.add({ sessionId, customerId: customer.id, role: "user",
content: message }, clock())
pipeline.push(`3. sessão atual ${session.id}: ${history.length} mensagem(ns) recente(s) no
histórico; mensagem do cliente gravada`)

// 4-5. recuperação seletiva
const { memories, pastSessions } = this.retrieveAll(customer, message, sessionId)
pipeline.push(`4. memórias: ${memories.total} ativa(s) → ${memories.selected.length} relevante(s),
${memories.rejected.length} descartada(s)`)
pipeline.push(`5. interações anteriores: ${pastSessions.total} sessão(ões) resumida(s) →
${pastSessions.selected.length} relacionada(s)`)

// 6. contexto
const context = buildContext({
  customer,
  history,
  currentMessage: message,
  memories: memories.selected,
  pastSessions: pastSessions.selected,
  withMemory: true,
  now: clock(),
  tokenBudget: config.contextTokenBudget,
})
pipeline.push(`6. contexto montado: ~${context.estTokens} tokens (orçamento
${context.tokenBudget})${context.trimmed.length ? `; cortes: ${context.trimmed.join(", ")}` : ""}`)

// 7-8. LLM e resposta
const generation = await this.generate(context)
pipeline.push(`7-8. LLM ${this.deps.llm.name}: ${generation.fallbackReason ? `falhou
(${generation.fallbackReason}) → resposta de fallback` : "respondeu"}${generation.parseWarning ? `
[${generation.parseWarning}]` : ""}`)

// 9. persistência da interação
const assistantMessage = repos.messages.add({ sessionId, customerId: customer.id, role:
"assistant", content: generation.response }, clock())
repos.sessions.touch(sessionId, clock())
repos.customers.touch(customer.id, clock())

```

```

    const citedMemoryIds = context.refs.filter((r) => r.kind === "memory" &&
generation.citedLabels.includes(r.label)).map((r) => (r.kind === "memory" ? r.memory.id : ""))
    repos.contextLog.add(
      {
        sessionId,
        messageId: assistantMessage.id,
        withMemory: true,
        memoryIds: context.refs.filter((r) => r.kind === "memory").map((r) => (r.kind === "memory" ?
r.memory.id : "")),
        memoriesCited: generation.fallbackReason ? null : citedMemoryIds,
        estTokens: context.estTokens,
        promptTokens: generation.usage?.promptTokens ?? null,
        completionTokens: generation.usage?.completionTokens ?? null,
        llm: this.deps.llm.name,
      },
      clock()
    )
    pipeline.push("9. resposta gravada e contexto registrado em context_log")

    // 10. extração/atualização de memória – falha aqui não afeta a resposta já dada.
    let extraction: TurnResult["extraction"]
    try {
      extraction = await memoryService.extractAndStore(customer, userMessage)
      const counts = extraction.writes.reduce<Record<string, number>>((acc, w) => ({ ...acc,
[w.action]: (acc[w.action] ?? 0) + 1 })), { })
      pipeline.push(`10. memória (${extraction.extractor}): ${Object.entries(counts).map(([k, v]) =>
`$${v} ${k}`).join(", ") || "nenhuma candidata"}`)
    } catch (error) {
      extraction = { error: (error as Error).message }
      pipeline.push(`10. memória: extração falhou (${(error as Error).message}); atendimento segue`)
    }

    return { customer, session: repos.sessions.get(sessionId)!, userMessage, assistantMessage,
memories, pastSessions, context, generation, citedMemoryIds, extraction, pipeline }
  }

  // Mesma pergunta, duas condições. Não grava nada: é uma demonstração.
  // sem memória: instrução + sessão atual + mensagem atual
  // com memória: instrução + cliente + memórias relevantes + interações anteriores + sessão atual +
mensagem atual
  async compare(customerId: string, question: string, options: { sessionId?: string } = {}):
Promise<ComparisonResult> {
    const { repos, clock, config } = this.deps
    const message = this.validate(question)
    const customer = repos.customers.get(customerId)
    if (!customer) throw new UserFacingError(`Cliente ${customerId} não encontrado.`)
    const history = options.sessionId ? repos.messages.lastBySession(options.sessionId,
config.historyMessages) : []
    const { memories, pastSessions } = this.retrieveAll(customer, message, options.sessionId)

    const base = { customer, history, currentMessage: message, memories: memories.selected,

```



```

pastSessions: pastSessions.selected, now: clock(), tokenBudget: config.contextTokenBudget }
  const ctxWithout = buildContext({ ...base, withMemory: false })
  const ctxWith = buildContext({ ...base, withMemory: true })
  const [genWithout, genWith] = [await this.generate(ctxWithout), await this.generate(ctxWith)]

  return {
    customer,
    question: message,
    memories,
    pastSessions,
    withoutMemory: { context: ctxWithout, generation: genWithout },
    withMemory: { context: ctxWith, generation: genWith },
  }
}
}

```

src/services/memoryService.ts

```

// Memory Service: recuperação seletiva (o que entra no contexto) e escrita controlada
// (o que vira memória). As regras de decisão ficam em utils/relevance.ts e memory/memoryPolicy.ts.

import { transaction } from "../../database/database.ts"
import type { Customer, Memory, MemoryCandidate, Message, Session } from "../../domain/types.ts"
import { type MemoryExtractor, RuleBasedExtractor } from "../../memory/extractors.ts"
import { decideWrite } from "../../memory/memoryPolicy.ts"
import { compareSlots, describeSlots, preferenceSlots, renderSlots } from
"../../memory/preferenceSlots.ts"
import type { Repositories } from "../../repositories/index.ts"
import type { Clock } from "../../utils/misc.ts"
import { combineScore, MIN_TEXTUAL, recencyScore, round, selectRelevant, textualRelevance, type
Rejected, type Scored } from "../../utils/relevance.ts"
import { normalize, topicOf } from "../../utils/text.ts"

export type RetrievalResult<T> = { selected: Scored<T>[]; rejected: Rejected<T>[]; total: number }

export type MemoryWrite =
  | { action: "created"; candidate: MemoryCandidate; memory: Memory }
  | { action: "reinforced"; candidate: MemoryCandidate; memory: Memory; similarity: number }
  | { action: "superseded"; candidate: MemoryCandidate; memory: Memory; previous: Memory[] }
  | { action: "rejected"; candidate: MemoryCandidate; reason: string }

export type ExtractionReport = {
  extractor: string
  writes: MemoryWrite[]
  skipped: { text: string; reason: string }[]
  warning?: string
}

// O nome do cliente não é evidência de relevância nem de duplicidade.

```

```

export function nameTokens(customer: Customer): string[] {
  return normalize(customer.name).split(/\s+/).filter(Boolean)
}

export const PAST_SESSION_MIN_SCORE = 0.3

export class MemoryService {
  private readonly repos: Repositories
  private readonly extractor: MemoryExtractor
  private readonly clock: Clock
  private readonly reaffirmation = new RuleBasedExtractor()

  constructor(repos: Repositories, extractor: MemoryExtractor, clock: Clock) {
    this.repos = repos
    this.extractor = extractor
    this.clock = clock
  }

  get extractorName(): string {
    return this.extractor.name
  }

  // Busca de memórias relevantes para a mensagem atual (score + portão + limite).
  retrieve(customer: Customer, query: string, options: { limit: number; minScore: number }):
  RetrievalResult<Memory> {
    const now = this.clock()
    const stop = nameTokens(customer)
    const active = this.repos.memories.listActive(customer.id)
    const scored: Scored<Memory>[] = active.map((m) => {
      const textual = textualRelevance(query, m.content, stop)
      const recency = round(recencyScore(m.updatedAt, now))
      return { item: m, textual, importance: m.importance, recency, score: combineScore(textual.score,
m.importance, recency) }
    })
    return { ...selectRelevant(scored, options), total: active.length }
  }

  // Interações anteriores relacionadas: resumos de sessões encerradas, pelo mesmo critério
  // textual (sem importância; score = 0,7·textual + 0,3·recência).
  //
  // A sessão só entra se trouxer algo que a MEMÓRIA não cobre. Se ela é a origem de uma
  // memória sobre o assunto da pergunta, fica de fora: ou a informação já está no contexto
  // (memória ativa → redundante), ou deixou de valer (memória substituída → obsoleta).
  // Sem essa regra, o resumo da sessão antiga reinjetava "reuniões pela manhã" depois de o
  // cliente ter mudado para "depois das 14h" (achado do teste do cenário 4).
  retrievePastSessions(customer: Customer, query: string, options: { limit: number; excludeSessionId?:
string }): RetrievalResult<Session> {
    if (options.limit <= 0) return { selected: [], rejected: [], total: 0 }
    const now = this.clock()
    const stop = nameTokens(customer)
    const sessions = this.repos.sessions.listSummarized(customer.id, options.excludeSessionId)

```

```

const covered: Rejected<Session>[] = []
const scored: Scored<Session>[] = []
const preferences = this.repos.memories.listAll(customer.id).filter((m) => m.type ===
"preference")
for (const s of sessions) {
  const textual = textualRelevance(query, `${s.title ?? ""}. ${s.summary ?? ""}`, stop)
  const recency = round(recencyScore(s.lastInteractionAt, now))
  const entry = { item: s, textual, importance: 0, recency, score: round(0.7 * textual.score + 0.3
* recency) }
  const overlapping = this.repos.memories.listBySourceSession(s.id).find((m) =>
textualRelevance(query, m.content, stop).score >= MIN_TEXTUAL)
  // 2ª camada, independente da proveniência (2ª revisão do Codex): se o resumo fala do
  // mesmo atributo de uma preferência memorizada (horário de reunião, formato de relatório...),
  // quem responde por esse atributo é a memória, com o valor mais recente – mesmo que a
  // extração não tenha reconhecido a frase daquela sessão.
  const sessionSlots = preferenceSlots(`${s.title ?? ""}. ${s.summary ?? ""}`)
  const sameAttribute = sessionSlots.size ? preferences.find((m) => compareSlots(sessionSlots,
preferenceSlots(m.content)).shared.length > 0) : undefined
  if (overlapping) {
    covered.push({ ...entry, reason: `já representada pela memória ${overlapping.id}
(${overlapping.status === "active" ? "ativa → redundante" : "substituída → obsoleta"}` })
  } else if (sameAttribute) {
    covered.push({ ...entry, reason: `trata de ${describeSlots(sessionSlots)}, atributo já coberto
pela memória ${sameAttribute.id} (vale o valor mais recente)` })
  } else scored.push(entry)
}
const picked = selectRelevant(scored, { limit: options.limit, minScore: PAST_SESSION_MIN_SCORE })
return { selected: picked.selected, rejected: [...picked.rejected, ...covered], total:
sessions.length }
}

// Depois de cada interação: extrai candidatas da mensagem do cliente e aplica a política.
async extractAndStore(customer: Customer, message: Message): Promise<ExtractionReport> {
  const existing = this.repos.memories.listActive(customer.id).map((m) => ({ type: m.type, content:
m.content }))
  const extraction = await this.extractor.extract({ message: message.content, existing })
  const source = { sessionId: message.sessionId, messageId: message.id }
  const writes = extraction.candidates.map((c) => this.store(customer, c, source))
  if (extraction.extractor !== this.reaffirmation.name) writes.push(...(await
this.recordReaffirmations(customer, message, writes, source)))
  return { extractor: extraction.extractor, writes, skipped: extraction.skipped, warning:
extraction.warning }
}

// Rede de segurança do extrator por LLM (achado da revisão do Codex): se o modelo omitir
// uma informação só porque ela já existe, a sessão deixaria de constar como fonte da memória,
// e o resumo dela poderia voltar ao prompt depois que a memória fosse substituída. 0
// detector por regras, determinístico, registra essas reafirmações. Ele só reforça memória
// já existente: nunca cria nem substitui nada no modo LLM.
private async recordReaffirmations(customer: Customer, message: Message, done: MemoryWrite[],
source: { sessionId: string; messageId: string }): Promise<MemoryWrite[]> {

```

```

const touched = new Set(done.flatMap((w) => (w.action === "rejected" ? [] : [w.memory.id])))
const out: MemoryWrite[] = []
for (const c of (await this.reaffirmation.extract({ message: message.content, existing: []
})).candidates) {
  const decision = decideWrite(c, this.repos.memories.listActive(customer.id),
nameTokens(customer))
  if (decision.action !== "reinforce" || touched.has(decision.target.id)) continue
  touched.add(decision.target.id)
  const memory = transaction(this.repos.db, () =>
this.repos.memories.reinforce(decision.target.id, decision.target.importance, source, this.clock()))
  out.push({ action: "reinforced", candidate: c, memory, similarity: decision.similarity })
}
return out
}

// Grava uma candidata passando pela política (portão → dedup → atualização → criação).
store(customer: Customer, candidate: MemoryCandidate, source: { sessionId: string | null; messageId:
string | null }): MemoryWrite {
  const now = this.clock()
  const active = this.repos.memories.listActive(customer.id)
  const decision = decideWrite(candidate, active, nameTokens(customer))
  const base = {
    customerId: customer.id,
    type: candidate.type,
    content: candidate.content.trim(),
    importance: candidate.importance,
    origin: candidate.origin,
    sourceSessionId: source.sessionId,
    sourceMessageId: source.messageId,
  }

  switch (decision.action) {
    case "reject":
      return { action: "rejected", candidate, reason: decision.reason }
    case "reinforce":
      return { action: "reinforced", candidate, memory: transaction(this.repos.db, () =>
this.repos.memories.reinforce(decision.target.id, candidate.importance, source, now)), similarity:
decision.similarity }
    case "supersede":
      return transaction(this.repos.db, () => {
        const content = decision.carried.size ? `${ensurePeriod(base.content)} Mantido da
preferência anterior: ${renderSlots(decision.carried)}.` : base.content
        const memory = this.repos.memories.insert({ ...base, content, topic: topicOf(content) },
now)
        for (const t of decision.targets) this.repos.memories.supersede(t.id, memory.id, now)
        return { action: "superseded" as const, candidate, memory, previous: decision.targets }
      })
    case "create":
      return { action: "created", candidate, memory: transaction(this.repos.db, () =>
this.repos.memories.insert({ ...base, topic: decision.topic ?? topicOf(base.content) }, now)) }
  }
}

```

```

    }
}

function ensurePeriod(text: string): string {
    const t = text.trim()
    return /[.!?...]*$/ .test(t) ? t : `${t}.`
}

```

src/utils/relevance.ts

```

// Scoring de relevância: decide QUAL memória entra no contexto.
//
// score = 0,60 · textual + 0,25 · importância + 0,15 · recência
//
// textual = max(conceitos em comum / conceitos da pergunta, termos da pergunta achados / termos da pergunta)
// recência = 0,5 ^ (idade em dias / 30) (meia-vida de 30 dias)
//
// Regra de corte (portão): a memória só entra se textual ≥ MIN_TEXTUAL **e** score ≥ minScore,
// e no máximo `limit` memórias. Importância e recência ordenam, mas nunca colocam sozinhas uma
// memória no contexto: sem relação com a pergunta, ela fica de fora (regra contra uso forçado).

import { concepts, terms } from "../text.ts"

export const WEIGHTS = { text: 0.6, importance: 0.25, recency: 0.15 } as const
export const MIN_TEXTUAL = 0.25
export const RECENCY_HALF_LIFE_DAYS = 30

export type TextualMatch = {
    score: number
    conceptScore: number
    termScore: number
    sharedConcepts: string[]
    sharedTerms: string[]
}

export function textualRelevance(query: string, doc: string, extraStopwords: Iterable<string> = []): TextualMatch {
    const qc = concepts(query)
    const dc = concepts(doc)
    const sharedConcepts = [...qc].filter((c) => dc.has(c))
    const conceptScore = qc.size ? sharedConcepts.length / qc.size : 0

    const stop = [...extraStopwords]
    const qt = terms(query, stop)
    const dt = terms(doc, stop)
    const sharedTerms = [...qt].filter((t) => dt.has(t))
    const termScore = qt.size ? sharedTerms.length / qt.size : 0

    return { score: Math.max(conceptScore, termScore), conceptScore, termScore, sharedConcepts,

```

```

sharedTerms }
}

export function recencyScore(isoDate: string, now: Date, halfLifeDays = RECENCY_HALF_LIFE_DAYS):
number {
  const ageDays = Math.max(0, (now.getTime() - new Date(isoDate).getTime()) / 86_400_000)
  return Math.pow(0.5, ageDays / halfLifeDays)
}

export type Scored<T> = {
  item: T
  score: number
  textual: TextualMatch
  importance: number
  recency: number
}

export function combineScore(textual: number, importance: number, recency: number): number {
  return round(WEIGHTS.text * textual + WEIGHTS.importance * importance + WEIGHTS.recency * recency)
}

export type Rejected<T> = Scored<T> & { reason: string }

// Aplica o portão e o limite. Devolve também as rejeitadas, com o motivo, para o modo debug.
export function selectRelevant<T>({
  scored: Scored<T>[],
  options: { limit: number; minScore: number; minTextual?: number }
}): { selected: Scored<T>[]; rejected: Rejected<T>[] } {
  const minTextual = options.minTextual ?? MIN_TEXTUAL
  const ordered = [...scored].sort((a, b) => b.score - a.score)
  const selected: Scored<T>[] = []
  const rejected: Rejected<T>[] = []
  for (const s of ordered) {
    if (s.textual.score < minTextual) rejected.push({ ...s, reason: `sem relação com a mensagem
(textual ${s.textual.score.toFixed(2)} < ${minTextual})` })
    else if (s.score < options.minScore) rejected.push({ ...s, reason: `score ${s.score.toFixed(2)} <
mínimo ${options.minScore}` })
    else if (selected.length >= options.limit) rejected.push({ ...s, reason: `acima do limite de
${options.limit} memórias` })
    else selected.push(s)
  }
  return { selected, rejected }
}

export function round(n: number): number {
  return Math.round(n * 100) / 100
}

```

src/memory/memoryPolicy.ts

```
// Política de escrita da memória. Funções puras: decidem, para cada candidata, se ela é
// descartada, se reforça uma memória existente (deduplicação), se substitui uma antiga
// (atualização) ou se vira uma memória nova.
//
// 1. Portão ..... tipo válido, texto específico ( $\geq 2$  termos de conteúdo OU um conceito
// de domínio, ex.: "login"), não é cumprimento, importância  $\geq 0,3$ .
// 2. Preferência com atributo reconhecido (horário/modalidade de reunião, formato/canal
// de relatório, canal de contato – ver preferenceSlots.ts):
// mesmo atributo, mesmo valor ..... reforça a existente (deduplicação)
// mesmo atributo, OUTRO valor ..... a antiga vira "superseded" (atualização);
// atributos antigos não mencionados são mantidos
// atributos diferentes ..... convivem ("pela manhã" + "por vídeo")
// 3. Demais memórias: mesmo tipo e Dice  $\geq 0,6$  nos termos → reforça (deduplicação).
// 4. Criação ..... nada acima se aplica.

import type { Memory, MemoryCandidate } from "../domain/types.ts"
import { isMemoryType } from "../domain/types.ts"
import { concepts, dice, isSmallTalk, terms, topicOf } from "../utils/text.ts"
import { compareSlots, describeSlots, type PreferenceSlot, preferenceSlots } from
"./preferenceSlots.ts"

export const MIN_IMPORTANCE = 0.3
export const MIN_CONTENT_TERMS = 2
export const DEDUP_THRESHOLD = 0.6
export const MAX_CONTENT_CHARS = 400

export type WriteDecision =
  | { action: "reject"; reason: string }
  | { action: "create"; topic: string | null }
  | { action: "reinforce"; target: Memory; similarity: number }
  | { action: "supersede"; targets: Memory[]; topic: string; carried: Map<PreferenceSlot, string> }

// Palavras que não distinguem uma memória de outra (o nome do cliente, "cliente", etc.).
function contentTerms(text: string, nameTokens: string[]): Set<string> {
  return terms(text, nameTokens)
}

export function gate(candidate: MemoryCandidate, nameTokens: string[] = []): string | null {
  if (!isMemoryType(candidate.type)) return `tipo inválido: ${String(candidate.type)}`
  const content = candidate.content?.trim() ?? ""
  if (!content) return "conteúdo vazio"
  if (content.length > MAX_CONTENT_CHARS) return `conteúdo acima de ${MAX_CONTENT_CHARS} caracteres`
  if (isSmallTalk(content)) return "cumprimento/agradecimento"
  if (contentTerms(content, nameTokens).size < MIN_CONTENT_TERMS && concepts(content).size === 0)
  return "pouco específica (poucos termos de conteúdo e nenhum assunto de negócio)"
  if (!(candidate.importance >= MIN_IMPORTANCE)) return `importância ${candidate.importance} abaixo de
  ${MIN_IMPORTANCE}`
  return null
}
```

```

export function decideWrite(candidate: MemoryCandidate, active: Memory[], nameTokens: string[] = []):
WriteDecision {
  const rejection = gate(candidate, nameTokens)
  if (rejection) return { action: "reject", reason: rejection }

  const topic = topicOf(candidate.content)
  const sameType = active.filter((m) => m.type === candidate.type)

  // Preferência com atributo reconhecido: a comparação é por atributo e valor, não por palavras.
  const slots = candidate.type === "preference" ? preferenceSlots(candidate.content) : new Map()
  if (slots.size) {
    const conflicts: Memory[] = []
    for (const m of sameType) {
      const cmp = compareSlots(slots, preferenceSlots(m.content))
      if (!cmp.shared.length) continue
      if (cmp.conflicting.length) conflicts.push(m)
      else if (!cmp.candidateOnly.length) return { action: "reinforce", target: m, similarity: 1 }
      // Mesmos valores, mas a candidata acrescenta um atributo ("PDF" → "PDF por e-mail"):
      // a nova é mais completa e substitui a antiga sem contradizê-la.
      else conflicts.push(m)
    }
    if (conflicts.length) {
      // Atributos das memórias substituídas que a mensagem nova NÃO mencionou continuam valendo
      // e são levados para a memória nova (3ª revisão do Codex: "Excel por e-mail" → "PDF"
      // perdia o envio por e-mail).
      const carried = new Map<PreferenceSlot, string>()
      for (const m of conflicts) {
        const old = preferenceSlots(m.content)
        for (const k of compareSlots(slots, old).existingOnly) if (!carried.has(k)) carried.set(k,
old.get(k!))
      }
      return { action: "supersede", targets: conflicts, topic: describeSlots(slots), carried }
    }
    return { action: "create", topic }
  }

  // Sem atributo reconhecido (ex.: "gosta de café especial", problemas, fatos): repetição
  // é detectada pela similaridade dos termos.
  const candTerms = contentTerms(candidate.content, nameTokens)
  let best: { memory: Memory; similarity: number } | null = null
  for (const m of sameType) {
    const similarity = dice(candTerms, contentTerms(m.content, nameTokens))
    if (!best || similarity > best.similarity) best = { memory: m, similarity }
  }
  if (best && best.similarity >= DEDUP_THRESHOLD) return { action: "reinforce", target: best.memory,
similarity: round(best.similarity) }
  return { action: "create", topic }
}

function round(n: number): number {

```



```
    return Math.round(n * 100) / 100
  }
```

src/memory/preferenceSlots.ts

```
// Atributo e valor de uma preferência. Decide quando uma preferência nova SUBSTITUI a antiga:
// só quando fala do MESMO atributo com OUTRO valor.
//
// "reuniões pela manhã" → meeting_time = periodo:manha
// "reuniões antes das 14h" → meeting_time = antes:14:00 (≠ depois:14:00)
// "reuniões depois das 14:30" → meeting_time = depois:14:30 (minutos contam)
// "reuniões por videoconferência" → meeting_mode = video (convive com o horário)
// "relatórios em Excel por e-mail" → report_format = excel · report_channel = email
// "não quero relatórios em PDF" → report_format = nao:pdf
// "sem vídeo, mas não pela manhã" → meeting_mode = nao:video · meeting_time = nao:periodo:manha
//
// A negação vale para o que vem DEPOIS dela no mesmo trecho da frase (trechos separados por
// pontuação, "mas" e "porém") e termina no primeiro valor de outro atributo:
// em "pela manhã sem vídeo" e em "sem vídeo pela manhã", só o vídeo é negado.
// É separado dos termos de busca de propósito: a busca descarta "antes", "depois" e "não"
// como stopwords, mas aqui essas palavras mudam o sentido (revisões adversariais do Codex).

import { normalize } from "../utils/text.ts"

export type PreferenceSlot = "meeting_time" | "meeting_mode" | "report_format" | "report_channel" |
"contact_channel"

const MEETING = /\b(reunia\w*|reunio\w*|meetings?|calls?|agenda\w*)\b/
const REPORT = /\b(relatorio\w*|reports?|dashboards?)\b/
const NEGATION = /\b(nao|sem|nunca|evit\w*)\b/
const CLAUSE_BREAK = /[.,;!?]|[\.:](?!\d)|\bmas\b|\bpore\b|\bpois\b/ // ":" e "." entre dígitos fazem
parte da hora (14:30)

const FORMAT: [RegExp, string][] = [
  [/bpdf\b/g, "pdf"],
  [/b(excel|xlsx?|planilha\w*)\b/g, "excel"],
  [/bcsv\b/g, "csv"],
  [/b(word|docx?)\b/g, "word"],
  [/bhtml\b/g, "html"],
]

const CHANNEL: [RegExp, string][] = [
  [/be-?mails?\b/g, "email"],
  [/bwhatsapp\b/g, "whatsapp"],
  [/b(telefone\w*|ligac\w*|ligar|liguem)\b/g, "telefone"],
  [/bsms\b/g, "sms"],
  [/b(portal|drive|sharepoint)\b/g, "portal"],
]

const MEETING_MODE: [RegExp, string][] = [
  [/bpresencia\w*\b/g, "presencial"],
  [/b(videoconferencia\w*|video\w*|online|remot\w*|teams|zoom|meet|virtua\w*)\b/g, "video"],
```

```

    [/b(telefone\w*|ligac\w*)\b/g, "telefone"],
  ]

// Hora com minutos: "14h" = "14:00" = "14 horas"; "14h30" = "14:30".
const CLOCK = String.raw`(\d{1,2})\s*(?:h\s*(\d{2})?\b|:(\d{2})\b|\s*horas?\b)`

type Found = { value: string; index: number }

function clock(h: string, m1?: string, m2?: string): string {
  return `${Number(h)}:${(m1 ?? m2 ?? "00").padStart(2, "0")}`
}

function findAll(clause: string, table: [RegExp, string][]): Found[] {
  return table.flatMap(([re, value]) => [...clause.matchAll(re)].map((m) => ({ value, index: m.index })))
}

function findTimes(clause: string): Found[] {
  const found: Found[] = []
  let rest = clause
  for (const m of clause.matchAll(new RegExp(String.raw`\b(antes|depois|apos|ate|a partir)\s+(?:d[aeo]s?\s+)?${CLOCK}`, "g"))) {
    const op = m[1] === "apos" || m[1] === "a partir" ? "depois" : m[1]
    found.push({ value: `${op}:${clock(m[2], m[3], m[4])}`, index: m.index })
    rest = rest.replace(m[0], " ".repeat(m[0].length)) // mesma posição para os demais
  }
  for (const m of rest.matchAll(new RegExp(String.raw`\b${CLOCK}`, "g"))) found.push({ value: `em:${clock(m[1], m[2], m[3])}`, index: m.index })
  for (const m of clause.matchAll(/b(manha|tarde|noite)\b/g)) found.push({ value: `periodo:${m[1]}`, index: m.index })
  return found
}

type Kind = "time" | "mode" | "format" | "channel"
const FINDERS: [Kind, (clause: string) => Found[]][] = [
  ["time", findTimes],
  ["mode", (c) => findAll(c, MEETING_MODE)],
  ["format", (c) => findAll(c, FORMAT)],
  ["channel", (c) => findAll(c, CHANNEL)],
]

// Alcance da negação (4ª revisão do Codex): ela vale para os valores que vêm DEPOIS dela no
// mesmo trecho, mas termina quando aparece um valor de OUTRO atributo. Em "sem vídeo pela
// manhã", só o vídeo é negado; em "não pela manhã nem à tarde", os dois horários são.
function scopedValues(text: string): Record<Kind, string[]> {
  const out: Record<Kind, Set<string>> = { time: new Set(), mode: new Set(), format: new Set(),
channel: new Set() }
  for (const clause of text.split(CLAUSE_BREAK)) {
    const values = FINDERS.flatMap(([kind, find]) => find(clause).map((f) => ({ ...f, kind })))
    const negations = [...clause.matchAll(new RegExp(NEGATION.source, "g"))].map((m) => m.index)
    const events = [...negations.map((index) => ({ index, neg: true as const })), ...values.map((v) =>

```

```

({ ...v, neg: false as const })).sort(
  (a, b) => a.index - b.index || (a.neg ? -1 : 1)
)
let active = false
let scopeKind: Kind | null = null
let scopeIndex = -1
for (const e of events) {
  if (e.neg) {
    active = true
    scopeKind = null
    continue
  }
  // Valores na mesma posição (ex.: "telefone" é modalidade e canal) compartilham o alcance.
  if (active && scopeKind !== null && e.kind !== scopeKind && e.index !== scopeIndex) active =
false
  if (active && scopeKind === null) {
    scopeKind = e.kind
    scopeIndex = e.index
  }
  out[e.kind].add(active ? `nao:${e.value}` : e.value)
}
}
return { time: [...out.time].sort(), mode: [...out.mode].sort(), format: [...out.format].sort(),
channel: [...out.channel].sort() }
}

// Mapa atributo → valor (valores ordenados e unidos por vírgula).
export function preferenceSlots(text: string): Map<PreferenceSlot, string> {
  const n = normalize(text).replace(/\b(bom dia|boa tarde|boa noite)\b/g, " ")
  const slots = new Map<PreferenceSlot, string>()
  const put = (slot: PreferenceSlot, values: string[]) => {
    if (values.length) slots.set(slot, values.join(","))
  }

  const v = scopedValues(n)
  const meeting = MEETING.test(n)
  const report = REPORT.test(n) || v.format.length > 0

  if (meeting) {
    put("meeting_time", v.time)
    put("meeting_mode", v.mode)
  }
  if (report) {
    put("report_format", v.format)
    put("report_channel", v.channel.filter((c) => !c.endsWith("telefone")))
  }
  if (!meeting && !report) put("contact_channel", v.channel)
  return slots
}

export type SlotComparison = { shared: PreferenceSlot[]; conflicting: PreferenceSlot[]; candidateOnly:

```

```

PreferenceSlot[]; existingOnly: PreferenceSlot[] }

export function compareSlots(candidate: Map<PreferenceSlot, string>, existing: Map<PreferenceSlot,
string>): SlotComparison {
  const shared = [...candidate.keys()].filter((k) => existing.has(k))
  return {
    shared,
    conflicting: shared.filter((k) => candidate.get(k) !== existing.get(k)),
    candidateOnly: [...candidate.keys()].filter((k) => !existing.has(k)),
    existingOnly: [...existing.keys()].filter((k) => !candidate.has(k)),
  }
}

export function describeSlots(slots: Map<PreferenceSlot, string>): string {
  return [...slots].map(([k, v]) => `${k}=${v}`).join(", ")
}

// Texto de um atributo, para consolidar numa memória nova os atributos antigos que a
// atualização não mencionou ("Excel por e-mail" → "PDF" mantém o envio por e-mail).
// O texto gerado é relido por preferenceSlots e produz o mesmo atributo e valor (testado).
const LABEL: Record<string, string> = {
  pdf: "PDF", excel: "Excel", csv: "CSV", word: "Word", html: "HTML",
  email: "e-mail", whatsapp: "WhatsApp", telefone: "telefone", sms: "SMS", portal: "portal",
  presencial: "presenciais", video: "por vídeo",
}

function renderValue(slot: PreferenceSlot, value: string): string {
  const neg = value.startsWith("nao:")
  const v = neg ? value.slice(4) : value
  let text: string
  if (slot === "meeting_time") {
    const [kind, ...rest] = v.split(":")
    const hour = rest.join(":").replace(/:00$/, "h")
    text = kind === "periodo" ? `pela ${rest[0]} === "manha" ? "manhã" : rest[0]}` : kind === "em" ?
`às ${hour}` : `${kind} das ${hour}`
  } else if (slot === "meeting_mode") text = v === "telefone" ? "por telefone" : LABEL[v] ?? v
  else if (slot === "report_format") text = `em ${LABEL[v] ?? v}`
  else text = `por ${LABEL[v] ?? v}`
  return neg ? `não ${text}` : text
}

const SLOT_PREFIX: Record<PreferenceSlot, string> = {
  meeting_time: "reuniões",
  meeting_mode: "reuniões",
  report_format: "relatórios",
  report_channel: "envio dos relatórios",
  contact_channel: "contato",
}

export function renderSlots(slots: Map<PreferenceSlot, string>): string {
  // Cada valor vira um trecho próprio (separado por ";"), para a negação de um não vazar no outro.

```

```

    return [...slots].flatMap(([slot, values]) => values.split(",").map((v) => `${SLOT_PREFIX[slot]}
    ${renderValue(slot, v)}`)).join("; ")
  }

```

src/services/contextService.ts

```

// Context Service: monta o prompt final, em seções nomeadas, dentro de um orçamento de tokens.
//
// system: SYSTEM INSTRUCTIONS
// CURRENT CUSTOMER ↴
// RELEVANT CUSTOMER MEMORY ↳ só no modo "com memória"
// RELATED PAST INTERACTIONS ⌋
// user/assistant: CURRENT SESSION (últimas N mensagens da sessão atual)
// user: CURRENT MESSAGE
//
// Se o total estimado passar do orçamento, corta nesta ordem: interações anteriores de menor
// score → memórias de menor score → mensagens mais antigas do histórico. Instrução e
// mensagem atual nunca são cortadas.

import type { Customer, Memory, Message, Session } from "../domain/types.ts"
import type { ChatMessage } from "../llm/llmClient.ts"
import { SYSTEM_INSTRUCTIONS } from "../llm/prompts.ts"
import { estimateTokens, formatDate } from "../utils/misc.ts"
import type { Scored } from "../utils/relevance.ts"

export type ContextRef =
  | { label: string; kind: "memory"; memory: Memory; score: number }
  | { label: string; kind: "session"; session: Session; score: number }

export type BuiltContext = {
  withMemory: boolean
  messages: ChatMessage[]
  refs: ContextRef[]
  sections: { name: string; tokens: number; items: number }[]
  estTokens: number
  tokenBudget: number
  trimmed: string[]
}

export type BuildContextInput = {
  customer: Customer
  history: Message[]
  currentMessage: string
  memories: Scored<Memory>[]
  pastSessions: Scored<Session>[]
  withMemory: boolean
  now: Date
  tokenBudget: number
}

```

```

export function renderMemoryLine(label: string, m: Memory): string {
  const mentions = m.mentions > 1 ? ` (mencionado ${m.mentions}x)` : ""
  return `[${label}] (${m.type} · ${formatDate(m.updatedAt)}) ${m.content}${mentions}`
}

export function renderSessionLine(label: string, s: Session): string {
  return `[${label}] (${formatDate(s.startedAt)}) ${s.title ?? "Atendimento"} – ${s.summary ?? ""}`
}

function customerSection(c: Customer, now: Date): string {
  return [
    "# CURRENT CUSTOMER",
    `Nome: ${c.name}`,
    `Empresa: ${c.company ?? "não informada"}`,
    ...(c.role ? [ `Cargo: ${c.role}` ] : []),
    `Data de hoje: ${formatDate(now.toISOString)}`,
  ].join("\n")
}

function compose(input: BuildContextInput, memories: Scored<Memory>[], sessions: Scored<Session>[],
history: Message[]) {
  const refs: ContextRef[] = []
  const blocks = [`# SYSTEM INSTRUCTIONS\n${SYSTEM_INSTRUCTIONS}`]
  const sections: BuiltContext["sections"] = [{ name: "SYSTEM INSTRUCTIONS", tokens:
estimateTokens(blocks[0]), items: 1 }]

  if (input.withMemory) {
    const cust = customerSection(input.customer, input.now)
    blocks.push(cust)
    sections.push({ name: "CURRENT CUSTOMER", tokens: estimateTokens(cust), items: 1 })

    const memLines = memories.map((s, i) => {
      const label = `M${i + 1}`
      refs.push({ label, kind: "memory", memory: s.item, score: s.score })
      return renderMemoryLine(label, s.item)
    })
    const mem = `# RELEVANT CUSTOMER MEMORY\n${memLines.length ? memLines.join("\n") : "(nenhuma
memória relevante para esta mensagem)}`
    blocks.push(mem)
    sections.push({ name: "RELEVANT CUSTOMER MEMORY", tokens: estimateTokens(mem), items:
memLines.length })

    if (sessions.length) {
      const sesLines = sessions.map((s, i) => {
        const label = `S${i + 1}`
        refs.push({ label, kind: "session", session: s.item, score: s.score })
        return renderSessionLine(label, s.item)
      })
      const ses = `# RELATED PAST INTERACTIONS\n${sesLines.join("\n")}`
      blocks.push(ses)
      sections.push({ name: "RELATED PAST INTERACTIONS", tokens: estimateTokens(ses), items:

```

```

sesLines.length })
    }
}

const messages: ChatMessage[] = [
  { role: "system", content: blocks.join("\n\n") },
  ...history.map((m) => ({ role: m.role, content: m.content }) as ChatMessage),
  { role: "user", content: input.currentMessage },
]

sections.push({ name: "CURRENT SESSION", tokens: history.reduce((t, m) => t +
estimateTokens(m.content), 0), items: history.length })
sections.push({ name: "CURRENT MESSAGE", tokens: estimateTokens(input.currentMessage), items: 1 })
const estTokens = messages.reduce((t, m) => t + estimateTokens(m.content), 0)
return { messages, refs, sections, estTokens }
}

export function buildContext(input: BuildContextInput): BuiltContext {
  let memories = input.withMemory ? [...input.memories].sort((a, b) => b.score - a.score) : []
  let sessions = input.withMemory ? [...input.pastSessions].sort((a, b) => b.score - a.score) : []
  let history = [...input.history]
  const trimmed: string[] = []

  let built = compose(input, memories, sessions, history)
  while (built.estTokens > input.tokenBudget) {
    if (sessions.length) {
      const dropped = sessions.pop()!
      trimmed.push(`interação anterior ${dropped.item.id} (score ${dropped.score})`)
    } else if (memories.length) {
      const dropped = memories.pop()!
      trimmed.push(`memória ${dropped.item.id} (score ${dropped.score})`)
    } else if (history.length) {
      const dropped = history.shift()!
      trimmed.push(`mensagem antiga ${dropped.id} do histórico`)
    } else break
    built = compose(input, memories, sessions, history)
  }

  return { withMemory: input.withMemory, ...built, tokenBudget: input.tokenBudget, trimmed }
}

// Versão legível do prompt, para o modo debug e para a demonstração.
export function renderContextForDebug(ctx: BuiltContext): string {
  const [system, ...rest] = ctx.messages
  const current = rest.at(-1)!
  const history = rest.slice(0, -1)
  return [
    system.content,
    `# CURRENT SESSION\n${history.length ? history.map((m) => `${m.role}: ${m.content}`).join("\n")} :
"(sessão sem mensagens anteriores)"`,
    `# CURRENT MESSAGE\n${current.content}`,
  ]
}

```

```
].join("\n\n")
}
```

src/database/migrations.ts

```
// Esquema do banco, em migrações numeradas. A versão aplicada fica em PRAGMA user_version,
// então reabrir um banco existente não recria nada e um banco antigo é atualizado em ordem.
```

```
export const MIGRATIONS: { version: number; description: string; sql: string }[] = [
  {
    version: 1,
    description: "clientes, sessões, mensagens, memórias e registro de contexto",
    sql: `
      CREATE TABLE customers (
        id TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        company TEXT,
        role TEXT,
        notes TEXT,
        created_at TEXT NOT NULL,
        last_interaction_at TEXT
      );

      CREATE TABLE sessions (
        id TEXT PRIMARY KEY,
        customer_id TEXT NOT NULL REFERENCES customers(id),
        title TEXT,
        summary TEXT,
        status TEXT NOT NULL CHECK (status IN ('active', 'closed')),
        started_at TEXT NOT NULL,
        last_interaction_at TEXT NOT NULL,
        ended_at TEXT
      );

      CREATE INDEX idx_sessions_customer ON sessions(customer_id, started_at);

      CREATE TABLE messages (
        id TEXT PRIMARY KEY,
        seq INTEGER NOT NULL, -- ordem estável dentro da sessão
        session_id TEXT NOT NULL REFERENCES sessions(id),
        customer_id TEXT NOT NULL REFERENCES customers(id),
        role TEXT NOT NULL CHECK (role IN ('user', 'assistant')),
        content TEXT NOT NULL,
        created_at TEXT NOT NULL
      );

      CREATE INDEX idx_messages_session ON messages(session_id, seq);

      CREATE TABLE memories (
        id TEXT PRIMARY KEY,
        customer_id TEXT NOT NULL REFERENCES customers(id),
        type TEXT NOT NULL CHECK (type IN ('preference', 'previous_problem', 'request',
```



```

'general_fact')),
    topic TEXT,
    content TEXT NOT NULL,
    importance REAL NOT NULL CHECK (importance BETWEEN 0 AND 1),
    status TEXT NOT NULL CHECK (status IN ('active', 'superseded')),
    superseded_by TEXT REFERENCES memories(id),
    source_session_id TEXT REFERENCES sessions(id),
    source_message_id TEXT REFERENCES messages(id),
    origin TEXT NOT NULL CHECK (origin IN ('rule_extractor', 'llm_extractor', 'manual')),
    mentions INTEGER NOT NULL DEFAULT 1,
    created_at TEXT NOT NULL,
    updated_at TEXT NOT NULL
);
CREATE INDEX idx_memories_customer ON memories(customer_id, status);

-- Evidência de cada chamada ao modelo: que memórias entraram e o tamanho do contexto.
CREATE TABLE context_log (
    id TEXT PRIMARY KEY,
    session_id TEXT NOT NULL REFERENCES sessions(id),
    message_id TEXT REFERENCES messages(id),
    with_memory INTEGER NOT NULL,
    memory_ids TEXT NOT NULL, -- JSON: memórias enviadas ao modelo
    memories_cited TEXT, -- JSON: memórias que o modelo declarou ter usado
    est_tokens INTEGER NOT NULL,
    prompt_tokens INTEGER,
    completion_tokens INTEGER,
    llm TEXT NOT NULL,
    created_at TEXT NOT NULL
);
\,
},
{
    version: 2,
    description: "proveniência das memórias (cada sessão/mensagem que criou ou reforçou uma memória)",
    sql: `
        -- Uma memória pode nascer numa sessão e ser reforçada (deduplicação) em outras. Sem esta
        -- tabela, só a sessão de origem ficava ligada à memória, e o resumo de uma sessão que
        -- reforçou uma preferência depois substituída voltava ao prompt como "interação anterior".
        CREATE TABLE memory_sources (
            memory_id TEXT NOT NULL REFERENCES memories(id),
            session_id TEXT REFERENCES sessions(id),
            message_id TEXT REFERENCES messages(id),
            action TEXT NOT NULL CHECK (action IN ('created', 'reinforced')),
            created_at TEXT NOT NULL
        );
        CREATE INDEX idx_memory_sources_session ON memory_sources(session_id);
        INSERT INTO memory_sources (memory_id, session_id, message_id, action, created_at)
            SELECT id, source_session_id, source_message_id, 'created', created_at FROM memories;
    `,
},

```

```
]
```

```
export const SCHEMA_VERSION = MIGRATIONS.at(-1)!.version
```